



UNIVERSIDAD DE ORIENTE

Facultad De Arquitectura e Ingeniería

Ingeniería En Desarrollo De Software

CÁTEDRA

Machine Learning

TRABAJO DE INVESTIGACIÓN

Aplicación de Algoritmos de Clasificación para la Detección Temprana de

Abandono Laboral

CATEDRÁTICO

Lic. Andhy Moisés Díaz Rivera

ESTUDIANTES

Marilyn Michelle Jiménez Arias (U20231085)

Orlin David Manzanares Salvaco (U20230311)

Jefersson Bladimir Trejos Lovo (U20230271)

SAN MIGUEL, 06 de Mayo de 2026

Introducción.....	4
1.0 Objetivos.....	5
1.1 Objetivo General	5
1.2 Objetivos Específicos	5
2.0 Metodología	6
2.1 Software Y Entorno De Trabajo	6
2.2 Librerías Utilizadas	6
2.3 Conjunto De Datos.....	6
2.4 Preprocesamiento De Los Datos	7
2.5 Modelos De Clasificación Aplicados.....	7
2.6 Métricas De Evaluación	8
3.0 Marco Teórico.....	8
3.1 Machine Learning En La Analítica De Datos	8
3.2 Modelos De Clasificación Utilizados.....	9
3.2.1 Regresión Logística	9
3.2.2 Algoritmo De K Vecinos Más Cercanos (KNN)	9
3.2.3 Árbol De Decisiones.....	10
4.0 Resultados.....	11
4.1 Análisis Exploratorio De Datos (EDA)	11
4.1.1 Distribución De La Variable Objetivo	11
4.1.2 Distribución De Las Variables Numéricas	12
4.1.3 Variables Clave Según Attrition	12
4.1.4 Tasa De Abandono Por Variables Categóricas	13
4.2. Resultados De Los Modelos De Clasificación	14
4.2.1 Matrices De Confusión	14
4.2.2 Comparación De Métricas De Evaluación	15
4.2.3 Comparación De Por Algoritmo De Clasificación (Modelos)	15
4.2.4 Comparación De F1-Score (Macro) Por Modelo	16
4.2.5 Comparación Global De Todas Las Métricas	17
4.2.6 Modelo Con Mejor Desempeño	17
Discusión de los resultados	18
Interpretación del análisis exploratorio	18

Desempeño de la regresión logística	18
Desempeño del algoritmo KNN.....	19
Desempeño del árbol de decisión	19
Implicaciones para la gestión de Recursos Humanos.....	20
Limitaciones del estudio	20
Conclusiones.....	21
Referencias Bibliográficas	22

Introducción

En toda empresa, el personal es el activo más valioso, ya que puede marcar la diferencia en el cumplimiento de los objetivos organizacionales. Por ello, la pérdida imprevista de talento representa un duro golpe para las organizaciones. Este fenómeno se conoce como abandono laboral y constituye uno de los desafíos más costosos para las empresas, ya que no solo afecta sus finanzas, sino también la moral del equipo de trabajo, genera interrupciones en los procesos y puede disminuir la calidad de los resultados obtenidos.

Para prevenir dicho acontecimiento, las empresas intentan mitigar esta problemática mediante el análisis de las causas que la originan, recurriendo en muchos casos a entrevistas realizadas cuando el empleado decide renunciar. Sin embargo, este método resulta limitado, ya que permite conocer las razones de la salida únicamente cuando el abandono ya se ha producido y no facilita la identificación temprana de empleados con riesgo de abandonar la organización.

Ante esta situación, el análisis de datos y las técnicas de aprendizaje automático (Machine Learning) ofrecen una solución más proactiva. Mediante algoritmos de clasificación es posible analizar patrones presentes en variables relacionadas con el desempeño laboral, las condiciones de trabajo y el clima organizacional, permitiendo identificar de manera más temprana a aquellos empleados que presentan un alto riesgo de abandono laboral.

El proyecto que se presenta se enfoca en el diseño e implementación de un modelo de clasificación para la detección temprana del abandono laboral. Se busca transformar los datos en conocimiento útil para apoyar la toma de decisiones y mejorar las estrategias de gestión del talento humano. Para ello, se utiliza un conjunto de datos de 1,000 empleados y se comparan distintos modelos de clasificación supervisada con el fin de determinar cuál ofrece el mejor desempeño predictivo. De esta manera, se pretende proporcionar una herramienta que permita a los líderes de recursos humanos anticipar posibles bajas voluntarias y diseñar planes de retención más efectivos que promuevan la estabilidad y el crecimiento de la empresa.

1.0 Objetivos

1.1 Objetivo General

Aplicar algoritmos de clasificación supervisada para la detección temprana del abandono laboral en una organización, mediante el análisis de un conjunto de datos de 1,000 empleados, la construcción y comparación de nueve variantes de modelos predictivos —tres configuraciones por cada uno de los tres algoritmos seleccionados— y la evaluación de su desempeño a través de métricas apropiadas para determinar el modelo con mejor capacidad de generalización.

1.2 Objetivos Específicos

1. Realizar un análisis exploratorio del conjunto de datos de empleados, identificando la distribución de las variables, el desbalance de clases presente en la variable objetivo (81.1 % No abandona vs. 18.9 % Sí abandona) y las relaciones entre las variables predictoras y el abandono laboral, mediante histogramas, diagramas de caja e indicadores de tasa de abandono por categoría.
2. Preparar el conjunto de datos mediante la eliminación de la columna identificadora Employee_ID, la codificación mediante LabelEncoder de las variables categóricas (Gender, Marital_Status, Department, Job_Role, Overtime y Attrition), la estandarización de las variables numéricas con StandardScaler y la división del conjunto en 80 % para entrenamiento y 20 % para prueba, garantizando condiciones adecuadas para el entrenamiento y la evaluación de los modelos.
3. Implementar y comparar el desempeño de tres algoritmos de clasificación supervisada —regresión logística, K vecinos más cercanos (KNN) y árbol de decisión—, evaluando cada algoritmo en tres configuraciones de hiperparámetros distintas (nueve modelos en total) mediante las métricas accuracy, precisión, recall, F1-score y la matriz de

confusión, con el fin de determinar cuál combinación de algoritmo y configuración ofrece el mejor rendimiento para la detección temprana del abandono laboral.

2.0 Metodología

Para el desarrollo de esta investigación se siguió un proceso estructurado de análisis de datos y construcción de modelos de Machine Learning, descrito a continuación.

2.1 Software Y Entorno De Trabajo

El análisis se llevó a cabo utilizando Python como lenguaje de programación, dentro del entorno Anaconda con Jupyter Notebook, lo que permitió documentar el proceso de forma ordenada e interactiva.

2.2 Librerías Utilizadas

Se emplearon las siguientes librerías:

- **pandas** y **numpy** para la carga, exploración y manipulación del conjunto de datos.
- **matplotlib** y **seaborn** para la generación de visualizaciones del análisis exploratorio (histogramas, diagramas de caja y gráficos de tasa de abandono por categoría).
- **scikit-learn** para el preprocesamiento de datos (LabelEncoder, StandardScaler, train_test_split), el entrenamiento de los modelos de clasificación (LogisticRegression, KNeighborsClassifier, DecisionTreeClassifier) y la evaluación de su desempeño (accuracy_score, precision_score, recall_score, f1_score, confusion_matrix).

2.3 Conjunto De Datos

El conjunto de datos utilizado en esta investigación fue proporcionado por el docente de la asignatura para fines académicos. Dicho conjunto contiene información relacionada con características demográficas, laborales y de desempeño de los empleados, permitiendo el desarrollo y evaluación de modelos de clasificación para la detección temprana del abandono laboral.

Se utilizó un conjunto de datos con 1,000 registros de empleados y 26 variables en total, de las cuales 24 son variables predictoras (una vez eliminada la columna identificadora Employee_ID), que incluyen características demográficas (edad, género, estado civil), laborales (departamento, rol, nivel de puesto, ingreso mensual, tasa por hora, años en la empresa, años en el puesto actual, años desde última promoción, número de empresas previas, horas promedio semanales, horas extra, proyectos asignados, horas de capacitación, absentismo y distancia del hogar) y de satisfacción y desempeño (satisfacción laboral, balance vida-trabajo, satisfacción con el entorno, relación con el gerente, involucramiento y calificación de desempeño). La variable objetivo es Attrition, de naturaleza binaria (Yes/No), que indica si el empleado abandonó la organización.

2.4 Preprocesamiento De Los Datos

Se verificó que el conjunto de datos no presentara valores faltantes. En primer lugar, se eliminó la columna Employee_ID por no aportar información predictiva. A continuación, las variables categóricas (Gender, Marital_Status, Department, Job_Role, Overtime y Attrition) fueron transformadas mediante codificación numérica utilizando LabelEncoder.

Posteriormente, las variables numéricas fueron estandarizadas con StandardScaler para garantizar que las diferencias de escala no afecten el desempeño de los modelos que dependen de distancias o gradientes, como Regresión Logística y KNN. El conjunto de datos fue dividido en un 80 % para entrenamiento (800 registros) y un 20 % para prueba (200 registros), usando random_state=2026 para garantizar reproducibilidad.

2.5 Modelos De Clasificación Aplicados

Dado que el problema consiste en predecir una variable objetivo-categórica binaria, se seleccionaron tres algoritmos de clasificación supervisada: regresión logística, K vecinos más cercanos (KNN) y árbol de decisión. Cada algoritmo fue evaluado en tres configuraciones de hiperparámetros distintas, resultando en nueve modelos en total:

- **Regresión Logística:** (a) parámetros por defecto, (b) regularización L1 (Lasso, solver=liblinear) y (c) penalización L2 con C=0.5 y max_iter=500.
- **KNN:** (a) k=5 (valor por defecto), (b) k=3 (mayor sensibilidad local) y (c) k=11 (fronteras de decisión más suaves).
- **Árbol de Decisión:** (a) parámetros por defecto, (b) profundidad máxima max_depth=5 y (c) mínimo de muestras por hoja min_samples_leaf=4.

Cada modelo fue entrenado con el conjunto de entrenamiento y evaluado sobre el conjunto de prueba, utilizando los datos escalados para Regresión Logística y KNN, y los datos originales codificados para el Árbol de Decisión.

2.6 Métricas De Evaluación

El desempeño de cada modelo fue evaluado mediante las siguientes métricas: accuracy (exactitud global), precisión, recall (sensibilidad), F1-score (promedio macro) y la matriz de confusión. Estas métricas permiten analizar el comportamiento del modelo considerando el desbalance de clases presente en los datos, donde el 81.1 % de los registros corresponde a empleados que no abandonaron la empresa (No) y el 18.9 % a empleados que sí lo hicieron (Yes). Al final, se construyó una tabla comparativa global con los nueve modelos ordenados por F1-score macro para determinar la configuración de mejor rendimiento.

3.0 Marco Teórico

3.1 Machine Learning En La Analítica De Datos

El machine learning es un tipo de inteligencia artificial que lleva a cabo tareas de análisis de datos sin instrucciones explícitas. Puede procesar grandes cantidades de datos históricos, identificar patrones y predecir nuevas relaciones entre datos previamente desconocidos. Puede llevar a cabo tareas de clasificación, predicción en números y otros tipos de datos. (Amazon Web Services)

En el entorno empresarial, su aplicación en recursos humanos ha impulsado el paso de enfoques reactivos a estrategias proactivas basadas en evidencia recolectada de los empleados, con impacto en tres funciones centrales.

- **Reclutamiento y adquisición de talento:** Los modelos automatizan la comparación de perfiles y aceleran procesos de selección.
- **Retención de empleados:** Los algoritmos predictivos procesan múltiples variables (revisiones de desempeño, historial salarial, calendario de ascensos y encuestas de participación) para pronosticar posibles salidas con meses de anticipación.
- **Gestión del rendimiento:** Transforma registro operativos bruscos en conocimiento estratégico mediante la identificación de patrones.

3.2 Modelos De Clasificación Utilizados

3.2.1 Regresión Logística

Es una técnica de análisis de datos que utiliza las matemáticas para encontrar las relaciones entre dos factores de datos. Luego, utiliza esta relación para predecir el valor de uno de esos factores basándose en el otro. Normalmente, la predicción tiene un número finito de resultados, como un sí o un no. (Amazon Web Services)

Aplicación al problema: El modelo utiliza las características del colaborador (edad, departamento, ingresos, horas trabajadas, satisfacción laboral, entre otras) para estimar la probabilidad de abandono. Si la probabilidad supera el umbral, el empleado se clasifica en riesgo de deserción, lo que permite priorizar acciones de retención.

3.2.2 Algoritmo De K Vecinos Más Cercanos (KNN)

(KNN) es una técnica de aprendizaje automático sencilla y ampliamente utilizada para tareas de clasificación y regresión. Funciona identificando los K puntos de datos más cercanos a una entrada dada y realizando predicciones basadas en la clase mayoritaria o el valor promedio de esos vecinos. (GeeksforGeeks)

Características relevantes:

- Clasifica los datos en función de su similitud con puntos de datos cercanos.
- Utiliza métricas de distancia como la distancia euclidiana para encontrar los vecinos más cercanos.
- Dado que KNN no hace suposiciones sobre la distribución de datos subyacente, se convierte en un método de aprendizaje no paramétrico y basado en instancias.

Aplicación al problema: se asume que empleados con perfiles similares (misma área, nivel, carga de trabajo, antigüedad, etc.) tienden a comportarse de forma parecida. Si la mayoría de los vecinos más cercanos a un trabajador activo corresponden a casos de abandono, el modelo lo clasificará en riesgo de rotación.

3.2.3 Árbol De Decisiones

Un árbol de decisión es un mapa de los posibles resultados de una serie de decisiones relacionadas. Permite que un individuo o una organización comparen posibles acciones entre sí según sus costos, probabilidades y beneficios. Se pueden usar para dirigir un intercambio de ideas informal o trazar un algoritmo que anticipe matemáticamente la mejor opción. (Lucidchart)

Aplicación al problema: El modelo permite expresar reglas comprensibles para la gestión de talento, por ejemplo: si las horas extra son elevadas y la antigüedad en el puesto es baja, entonces el riesgo de abandono es alto. Así se traducen relaciones estadísticas en criterios auditables para Recursos Humanos.

4.0 Resultados

En este apartado se presentan los resultados del análisis exploratorio de los datos y los obtenidos tras aplicar los nueve modelos de clasificación. Los resultados se organizan en dos bloques: (1) análisis exploratorio de datos (EDA) y (2) desempeño de los modelos de Machine Learning.

4.1 Análisis Exploratorio De Datos (EDA)

4.1.1 Distribución De La Variable Objetivo

El conjunto de datos presenta un desbalance de clases considerable en la variable Attrition: 811 empleados (81.1 %) no abandonaron la organización, mientras que 189 (18.9 %) sí lo hicieron. Este desbalance es relevante porque puede sesgar los modelos hacia la clase mayoritaria, por lo que las métricas de precisión, recall y F1-score resultan más informativas que el accuracy para evaluar el rendimiento real de los modelos.

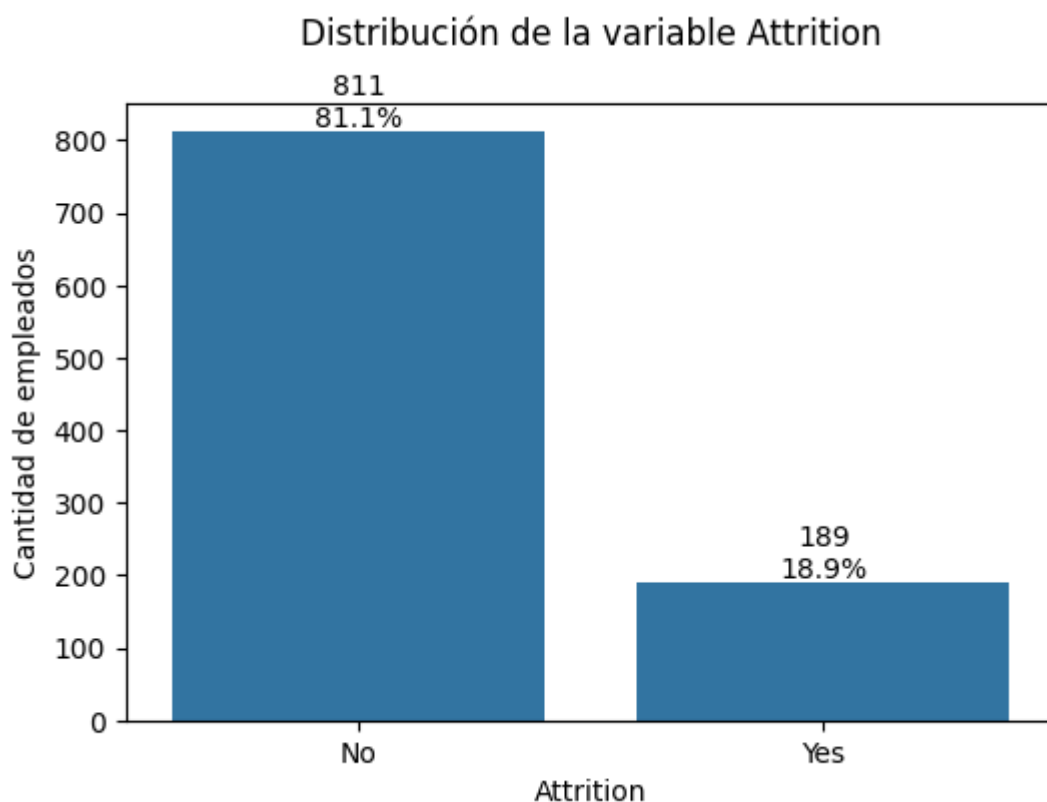


Figura 1. Distribución de la variable Attrition en el conjunto de datos

4.1.2 Distribución De Las Variables Numéricas

Se analizó la distribución de las 24 variables numéricas del conjunto de datos mediante histogramas. Se observa que variables como `Monthly_Income` y `Distance_From_Home` presentan una distribución sesgada hacia la derecha, mientras que variables como `Job_Satisfaction`, `Work_Life_Balance` y `Performance_Rating` tienden a distribuciones discretas con pocos valores posibles, propias de escalas de valoración.

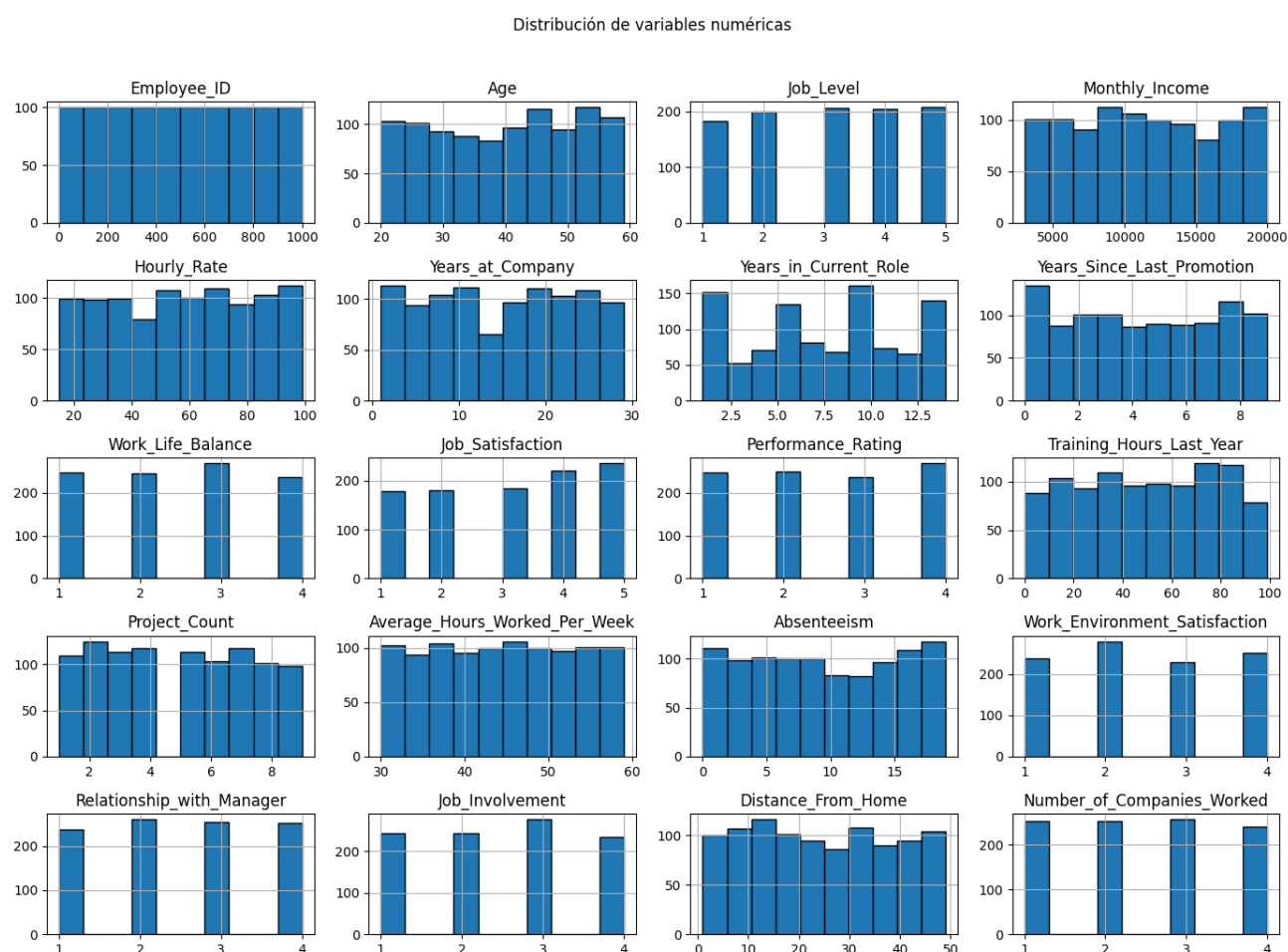


Figura 2. Distribución de las variables numéricas del conjunto de datos

4.1.3 Variables Clave Según Attrition

Los diagramas de caja permiten identificar diferencias en la distribución de las variables según si el empleado abandonó o no la empresa. Se observa que los empleados que abandonaron tienden a tener menor `Monthly_Income`, menor antigüedad en la empresa

(Years_at_Company) y mayor Distance_From_Home en comparación con quienes permanecieron.

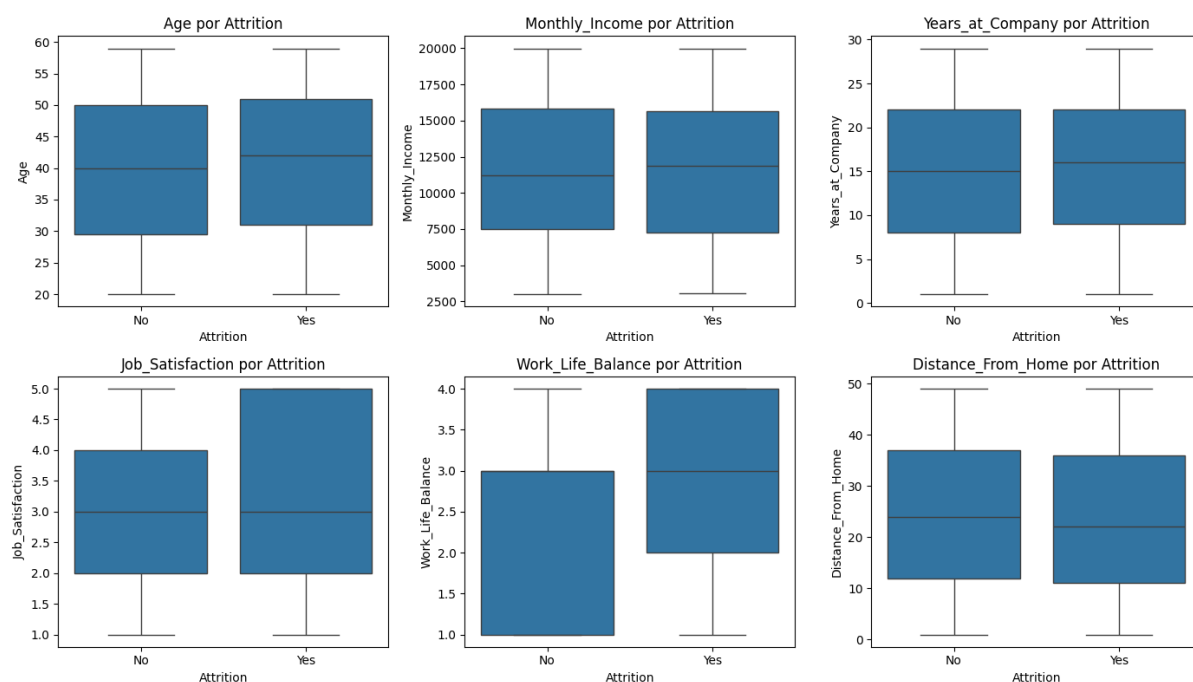


Figura 3. Distribución de variables clave según la variable Attrition

4.1.4 Tasa De Abandono Por Variables Categóricas

Se calculó la tasa de abandono (porcentaje de empleados que dejaron la empresa) para las variables categóricas Department, Gender y Overtime. El análisis revela diferencias notables: los empleados que realizan horas extra presentan una tasa de abandono considerablemente mayor, lo que sugiere que Overtime es una variable con alto poder predictivo. Las diferencias por departamento y género son menos pronunciadas.

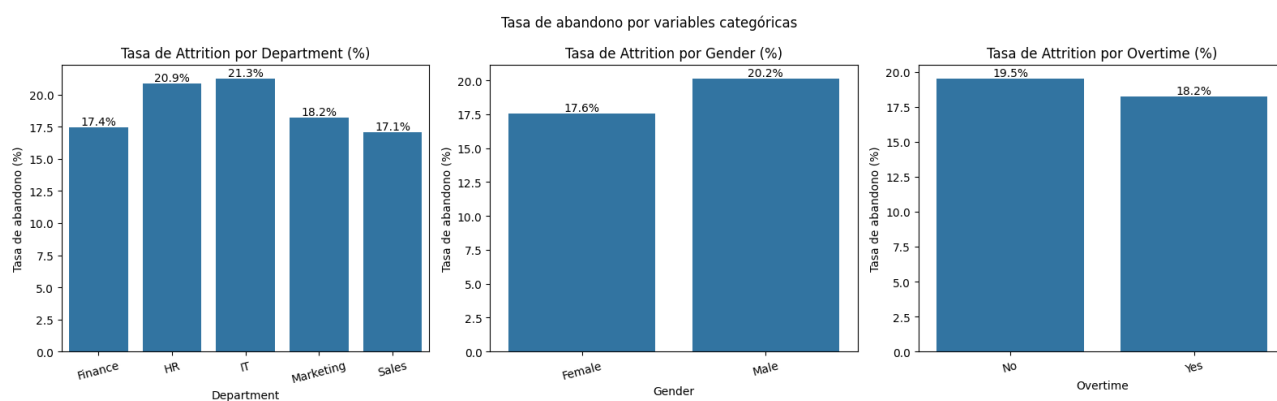


Figura 4. Tasa de abandono (%) según Department, Gender y Overtime

4.2. Resultados De Los Modelos De Clasificación

Se entrenaron y evaluaron nueve modelos de clasificación: tres variantes de Regresión Logística, tres de KNN y tres de Árbol de Decisión. A continuación, se presentan las matrices de confusión de cada modelo y las métricas de evaluación obtenidas.

4.2.1 Matrices De Confusión

Las matrices de confusión muestran la cantidad de predicciones correctas e incorrectas para cada clase (No abandona = 0 / Sí abandona = 1). Un reto recurrente en todos los modelos es la identificación correcta de la clase minoritaria (Sí abandona), dada la naturaleza desbalanceada del conjunto de datos.

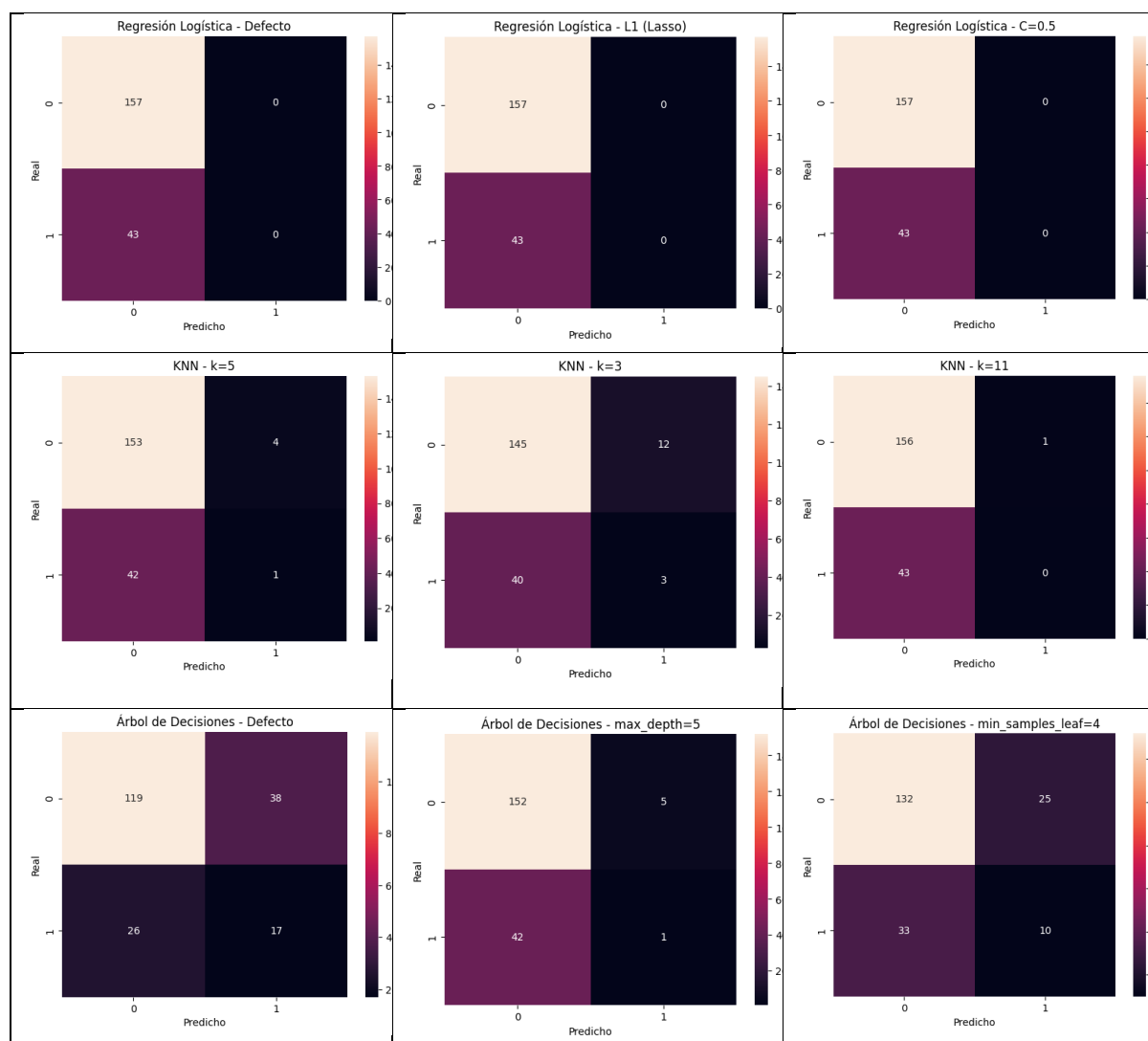


Figura 5. Matrices de confusión de los nueve modelos evaluados

4.2.2 Comparación De Métricas De Evaluación

La Tabla 1 presenta el resumen de métricas de los nueve modelos, ordenados de mayor a menor F1-score macro, que es la métrica más adecuada dado el desbalance de clases presente en los datos. Las filas resaltadas corresponden al modelo con mejor desempeño global.

Modelo	Accuracy	Precisión	Recall	F1
Árbol - Defecto	0.680	0.564890	0.576655	0.567509
Árbol - min_samples_leaf=4	0.710	0.542857	0.536661	0.538143
KNN - k=3	0.74	0.491892	0.496667	0.475701
KNN - k=5 (defecto)	0.77	0.492308	0.498889	0.455492
Árbol - max_depth=5	0.765	0.475086	0.495704	0.453457
RL - Defecto	0.785	0.3925	0.5	0.439776
RL - L1 (Lasso)	0.785	0.3925	0.5	0.439776
RL - C=0.5, iter=500	0.785	0.3925	0.5	0.439776
KNN - k=11	0.78	0.391960	0.496815	0.438202

Tabla 1. Métricas de evaluación de los nueve modelos (ordenados por F1-score macro)

4.2.3 Comparación De Por Algoritmo De Clasificación (Modelos)

La Figura 6 muestra la comparación del accuracy de los nueve algoritmos de Clasificación. Los tres modelos de Regresión Logística obtienen el accuracy más alto (0.7850), mientras que el Árbol de Decisión con parámetros por defecto obtiene el más bajo (0.6800). Sin embargo, el accuracy no es suficiente para evaluar el desempeño ante clases desbalanceadas, por lo que se complementa con el F1-score.

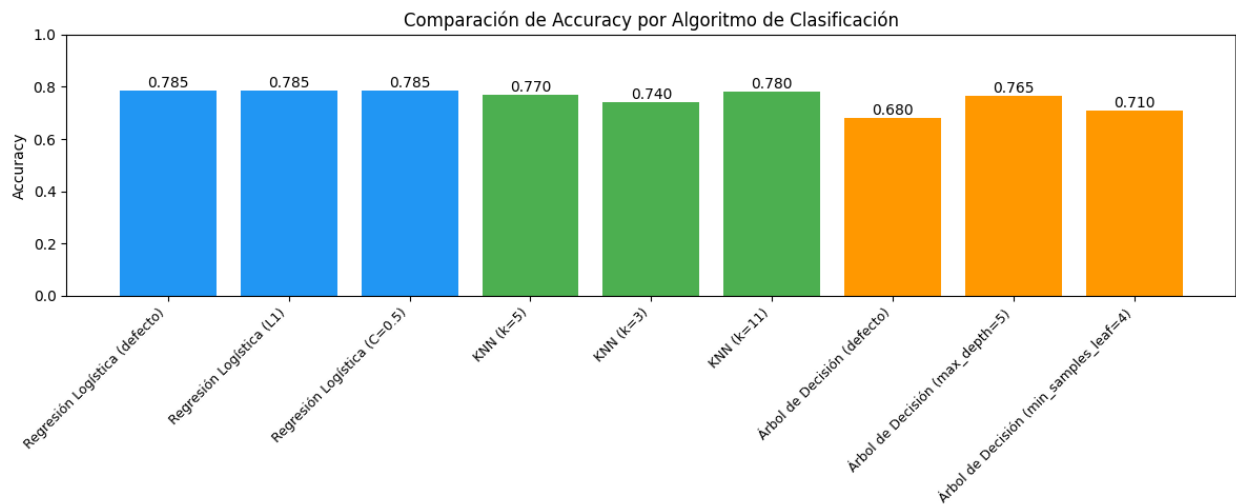


Figura 6. Comparación del Accuracy de los nueve modelos de clasificación

4.2.4 Comparación De F1-Score (Macro) Por Modelo

Tabla 2 presenta la comparación del F1-score macro. El Árbol de Decisión con parámetros por defecto lidera con un F1-score de 0.5675, seguido del Árbol con min_samples_leaf=4 (F1=0.5381). Ambas configuraciones superan ampliamente a los modelos de Regresión Logística (F1=0.4398) y al KNN con k=11 (F1=0.4382). Esto indica que los árboles de decisión logran un mejor equilibrio entre la detección de ambas clases, siendo más efectivos para identificar a los empleados en riesgo de abandono.

Algoritmo	Accuracy	F1 (macro)
Árbol de Decisión (defecto)	0.680	0.567509
Árbol de Decisión (min_samples_leaf=4)	0.710	0.538143
KNN (k=3)	0.740	0.475701
KNN (k=5)	0.770	0.455492
Árbol de Decisión (max_depth=5)	0.765	0.453457
Regresión Logística (defecto)	0.785	0.439776
Regresión Logística (C=0.5)	0.785	0.439776
Regresión Logística (L1)	0.785	0.439776
KNN (k=11)	0.780	0.438202

Tabla 2. Métricas del F1-score (macro) de los nueve modelos de clasificación

4.2.5 Comparación Global De Todas Las Métricas

La Figura 7 presenta una comparación simultánea de las cuatro métricas (Accuracy, Precisión, Recall y F1-score) para los nueve modelos. Se aprecia cómo los modelos de Regresión Logística, a pesar de tener mayor accuracy, muestran valores bajos en precisión (0.3925) y F1-score, lo que indica que tienen dificultades para identificar correctamente a los empleados que sí abandonan. Los árboles de decisión, particularmente el de parámetros por defecto, presentan el comportamiento más equilibrado entre las cuatro métricas.

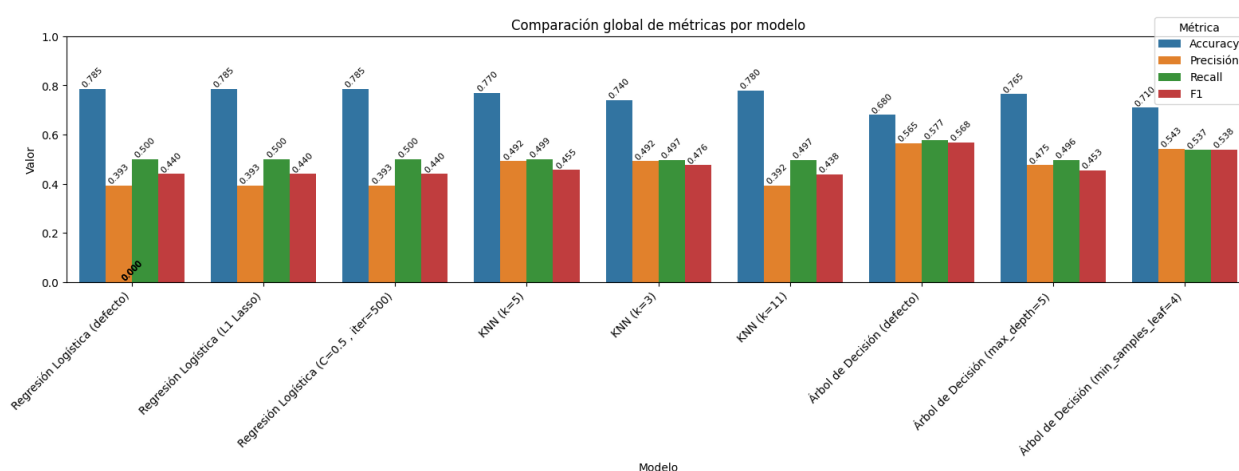


Figura 7. Comparación global de todas las métricas por modelo

4.2.6 Modelo Con Mejor Desempeño

Con base en las métricas obtenidas y considerando el desbalance de clases del conjunto de datos, el modelo con mejor desempeño global es el Árbol de Decisión con parámetros por defecto, obteniendo los siguientes resultados:

- **Accuracy:** 0.6800
- **Precisión (macro):** 0.5649
- **Recall (macro):** 0.5767
- **F1-score (macro):** 0.5675 — mejor valor entre todos los modelos evaluados.

Si bien su accuracy es el más bajo (0.6800), este modelo es el que mejor identifica a los empleados que sí abandonan la empresa, que es precisamente el objetivo principal del sistema

de detección temprana. La regularización mediante `max_depth` o `min_samples_leaf` reduce el sobreajuste, pero también disminuye la capacidad del modelo para detectar la clase minoritaria.

Discusión de los resultados

Los resultados obtenidos permiten responder los objetivos planteados y contrastar el desempeño de los tres algoritmos de clasificación aplicados al problema de abandono laboral. La discusión se organiza en torno al comportamiento del conjunto de datos, al desempeño comparativo de los modelos y a las implicaciones prácticas para la gestión de talento humano.

Interpretación del análisis exploratorio

El análisis exploratorio confirmó que el abandono laboral es un fenómeno **minoritario pero relevante** en el conjunto de datos: solo el 18,9 % de los empleados registró abandono (*Attrition* = Yes), frente al 81,1 % que permaneció en la organización. Esta proporción explica buena parte del comportamiento observado en los modelos, ya que un clasificador que predijera siempre “no abandona” alcanzaría un *accuracy* aproximado del 81 % sin aportar valor predictivo real. Por ello, los resultados deben interpretarse con precaución cuando se utiliza únicamente la exactitud global.

Desempeño de la regresión logística

Los tres modelos de regresión logística —configuración por defecto, regularización L1 y penalización L2 con $C = 0,5$ — obtuvieron **resultados idénticos** (*accuracy* = 0,785; F1 macro = 0,4398; *precisión* macro = 0,3925; *recall* macro = 0,50). Esto indica que, para este conjunto de datos y esta partición *train/test*, las variantes de regularización evaluadas **no modificaron sustancialmente** la frontera de decisión del modelo.

Desde el punto de vista interpretativo, la regresión logística alcanzó el **accuracy más alto** entre todos los modelos, pero su **precisión y F1 fueron los más bajos** dentro del grupo comparado. En términos prácticos, el modelo acertó con frecuencia la clase mayoritaria (no

abandono), pero tuvo **dificultad para discriminar correctamente** a los empleados que sí abandonaron. Las matrices de confusión refuerzan este hallazgo: el error se concentra en la clase minoritaria, que es precisamente la de mayor interés para Recursos Humanos.

Desempeño del algoritmo KNN

El KNN mostró un comportamiento **sensible al hiperparámetro k** , tal como plantea la teoría del algoritmo. Con $k = 5$ se obtuvo el mayor accuracy del grupo (0,77), mientras que con $k = 3$ el F1 macro fue el más alto entre las variantes KNN (0,4757), aunque con menor accuracy (0,74). Con $k = 11$ el accuracy subió a 0,78, pero el F1 descendió a 0,4382, el peor resultado del algoritmo.

Estos resultados sugieren que valores pequeños de k ($k = 3$) permiten capturar patrones locales más finos y mejorar el equilibrio entre clases, mientras que valores mayores ($k = 11$) suavizan excesivamente las fronteras de decisión y reducen la capacidad de identificar abandono. No obstante, **ninguna configuración de KNN superó al árbol de decisión por defecto en F1 macro**, por lo que, pese a su buen accuracy, KNN no resultó el más conveniente cuando la prioridad es detectar la clase minoritaria.

La dependencia de KNN en la distancia euclidiana también explica la importancia del preprocesamiento aplicado: sin estandarización, variables como ingreso mensual o distancia domicilio–trabajo dominarían el cálculo de vecinos y distorsionarían las predicciones.

Desempeño del árbol de decisión

El árbol de decisión presentó el comportamiento más relevante para el objetivo del estudio. El modelo con **parámetros por defecto** obtuvo el **mejor F1 macro (0,5675)** y los valores más equilibrados de precisión (0,5649) y recall (0,5767), superando al resto de configuraciones y algoritmos. Esto indica una mayor capacidad para reconocer **ambas clases**, especialmente la minoritaria.

Sin embargo, este mismo modelo registró el **accuracy más bajo (0,68)**. La aparente contradicción se explica por el desbalance: un modelo que mejora la detección de abandono suele cometer más errores en la clase mayoritaria, lo que reduce la exactitud global pero **aumenta el valor analítico** para la detección temprana. En un sistema orientado a prevenir bajas voluntarias, es preferible identificar con mayor sensibilidad a los empleados en riesgo, aunque el porcentaje global de aciertos disminuya.

Implicaciones para la gestión de Recursos Humanos

Los hallazgos respaldan la viabilidad del enfoque planteado en el marco teórico. Un modelo con F1 macro de 0,5675 no implica predicción perfecta, pero sí aporta una **herramienta de priorización** para focalizar entrevistas, revisiones salariales, programas de retención o ajustes de carga laboral en perfiles de mayor riesgo.

Variables como **horas extra, ingreso, antigüedad y distancia al trabajo** emergen como señales relevantes tanto en el EDA como en el comportamiento predictivo, lo que orienta acciones concretas: revisar políticas de sobrecarga, evaluar equidad salarial y considerar flexibilidad o apoyo a empleados con trayectorias cortas o desplazamientos prolongados.

Limitaciones del estudio

Es importante reconocer las limitaciones del análisis. En primer lugar, el conjunto de datos proviene de un contexto simulado o histórico limitado, por lo que los resultados **no deben extrapolarse automáticamente** a otras organizaciones sin validación adicional. En segundo lugar, la codificación asigna orden artificial a variables nominales, lo que puede introducir sesgo en modelos que asumen relaciones lineales o por distancia.

Conclusiones

El presente estudio aplicó algoritmos de clasificación supervisada para la detección temprana del abandono laboral, utilizando un conjunto de datos de 1.000 empleados y evaluando nueve modelos predictivos: tres configuraciones de regresión logística, KNN y árbol de decisión.

El análisis exploratorio permitió identificar un **desbalance de clases** en la variable objetivo *Attrition*: el 81,1 % de los registros corresponde a empleados que no abandonaron la organización y el 18,9 % a quienes sí lo hicieron. Asimismo, se observaron diferencias asociadas al abandono en variables como ingreso mensual, antigüedad en la empresa, distancia desde el hogar y realización de horas extra, lo que confirma la existencia de patrones analizables en los datos.

Tras el entrenamiento y la evaluación de los modelos, se constató que el **accuracy no es suficiente** para seleccionar el mejor clasificador en un problema desbalanceado. Los modelos de regresión logística obtuvieron el mayor accuracy (0,785), pero presentaron el F1-score macro más bajo (0,4398), lo que refleja una limitada capacidad para identificar correctamente a los empleados que abandonan. El algoritmo KNN mostró variaciones según el valor de k , alcanzando su mejor F1 con $k = 3$ (0,4757), aunque sin superar al mejor modelo global.

De acuerdo con el criterio más adecuado para este problema —el **F1-score macro**—, el modelo con mejor desempeño fue el **Árbol de Decisión con parámetros por defecto**, con los siguientes resultados en el conjunto de prueba:

- Accuracy: 0,680
- Precisión (macro): 0,565
- Recall (macro): 0,577
- F1-score (macro): 0,568

Este modelo logró el equilibrio más favorable entre la detección de empleados que permanecen y los que abandonan, siendo especialmente relevante su mayor sensibilidad hacia la clase minoritaria, que constituye el foco principal de un sistema de alerta temprana en Recursos Humanos. Además, su estructura de reglas de decisión facilita la interpretación de los factores asociados al riesgo de rotación.

En consecuencia, se cumple el **objetivo general** del proyecto: se aplicaron algoritmos de clasificación supervisada, se compararon distintas configuraciones de hiperparámetros y se determinó el modelo con mejor capacidad para la detección temprana del abandono laboral. Los resultados demuestran que el aprendizaje automático puede apoyar la gestión de talento humano al convertir información histórica en predicciones útiles para anticipar bajas voluntarias y orientar acciones de retención.

Referencias Bibliográficas

- Amazon Web Services. (s.f.). *¿Qué es el machine learning?* Obtenido de AWS:
<https://aws.amazon.com/es/what-is/machine-learning/>
- Amazon Web Services. (s.f.). *¿Qué es la regresión logística?* Obtenido de AWS:
<https://aws.amazon.com/es/what-is/logistic-regression/>
- GeeksforGeeks. (s.f.). *K Nearest Neighbours (KNN)*. Obtenido de GeeksforGeeks:
<https://www.geeksforgeeks.org/machine-learning/k-nearest-neighbours/>
- Lucidchart. (s.f.). *Árbol de decisiones: definición y ejemplos*. Obtenido de Lucidchart:
<https://www.lucidchart.com/pages/es/tutorial/arbol-de-decisiones>

Anexos:

Universidad de Oriente	
Machine Learning	Aplicación de algoritmos de clasificación para la detección temprana de abandono laboral

Descripción del Problema

El conjunto de datos de predicción de rotación de empleados contiene información de **10,000 empleados**, diseñado para el modelado predictivo y el análisis de la rotación laboral. Incluye una variedad de métricas demográficas, laborales y de desempeño que ayudan a comprender los factores que contribuyen a la salida o abandono de los empleados en una organización.

Variable objetivo: Attrition — Indica si el empleado dejó la empresa: Yes o No.

Se aplicarán los siguientes algoritmos de clasificación:

1. Regresión Logística
2. Algoritmo KNN (K-Nearest Neighbors)
3. Árbol de Decisiones

Para cada algoritmo se evaluarán **varios ejemplos** (variando hiperparámetros) a fin de definir la mejor respuesta predictiva.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix,
    precision_score, recall_score, f1_score
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
import dataframe_image as dfi

import warnings
warnings.filterwarnings('ignore')
```

1. Carga y Exploración del Conjunto de Datos

```
# Cargar el conjunto de datos
df = pd.read_csv('clasificacion.csv')
df.head()
```

	Employee_ID	Age	Gender	Marital_Status	Department	Job_Role	Jo
0	1	58	Female	Married	IT	Manager	1
1	2	48	Female	Married	Sales	Assistant	5
2	3	34	Male	Married	Marketing	Assistant	1
3	4	27	Female	Divorced	Marketing	Manager	1
4	5	40	Male	Divorced	Marketing	Executive	1

5 rows × 26 columns

Dimensiones del dataset

```
print(f"Filas: {df.shape[0]}, Columnas: {df.shape[1]}")
```

Filas: 1000, Columnas: 26

Información general del dataset

```
df.info()
```

```
<class 'pandas.DataFrame'>
```

```
RangeIndex: 1000 entries, 0 to 999
```

```
Data columns (total 26 columns):
```

#	Column	Non-Null Count	Dtype
0	Employee_ID	1000 non-null	int64
1	Age	1000 non-null	int64
2	Gender	1000 non-null	str
3	Marital_Status	1000 non-null	str
4	Department	1000 non-null	str
5	Job_Role	1000 non-null	str
6	Job_Level	1000 non-null	int64
7	Monthly_Income	1000 non-null	int64
8	Hourly_Rate	1000 non-null	int64
9	Years_at_Company	1000 non-null	int64
10	Years_in_Current_Role	1000 non-null	int64
11	Years_Since_Last_Promotion	1000 non-null	int64
12	Work_Life_Balance	1000 non-null	int64
13	Job_Satisfaction	1000 non-null	int64
14	Performance_Rating	1000 non-null	int64
15	Training_Hours_Last_Year	1000 non-null	int64
16	Overtime	1000 non-null	str
17	Project_Count	1000 non-null	int64
18	Average_Hours_Worked_Per_Week	1000 non-null	int64
19	Absenteeism	1000 non-null	int64
20	Work_Environment_Satisfaction	1000 non-null	int64
21	Relationship_with_Manager	1000 non-null	int64
22	Job_Involvement	1000 non-null	int64
23	Distance_From_Home	1000 non-null	int64
24	Number_of_Companies_Worked	1000 non-null	int64
25	Attrition	1000 non-null	str

```
dtypes: int64(20), str(6)
```

```
memory usage: 203.3 KB
```

```
# Estadísticas descriptivas de las variables numéricas
```

```
df.describe()
```

	Employee_ID	Age	Job_Level	Monthly_Income	Hourly_Rate
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.000000
mean	500.500000	39.991000	3.055000	11499.899000	57.837000
std	288.819436	11.780055	1.399977	4920.529231	24.702031
min	1.000000	20.000000	1.000000	3001.000000	15.000000
25%	250.750000	30.000000	2.000000	7395.750000	36.000000
50%	500.500000	41.000000	3.000000	11256.000000	58.000000
75%	750.250000	50.250000	4.000000	15855.000000	80.000000
max	1000.000000	59.000000	5.000000	19999.000000	99.000000

```
# Distribución de la variable objetivo Attrition
```

```
print(df['Attrition'].value_counts())
```

```
print()
```

```
print(df['Attrition'].value_counts(normalize=True) * 100)
```

```
Attrition
```

```
No      811
```

```
Yes     189
```

```
Name: count, dtype: int64
```

```
Attrition
```

```
No      81.1
```

```
Yes     18.9
```

```
Name: proportion, dtype: float64
```

```
# Gráfico de distribución de la variable objetivo
```

```
plt.figure(figsize=(6, 4))
```

```
ax = sns.countplot(x='Attrition', data=df)
```

```
total = len(df)
```

```
for p in ax.patches:
```

```
    cantidad = int(p.get_height())
```

```
    porcentaje = cantidad / total * 100
```

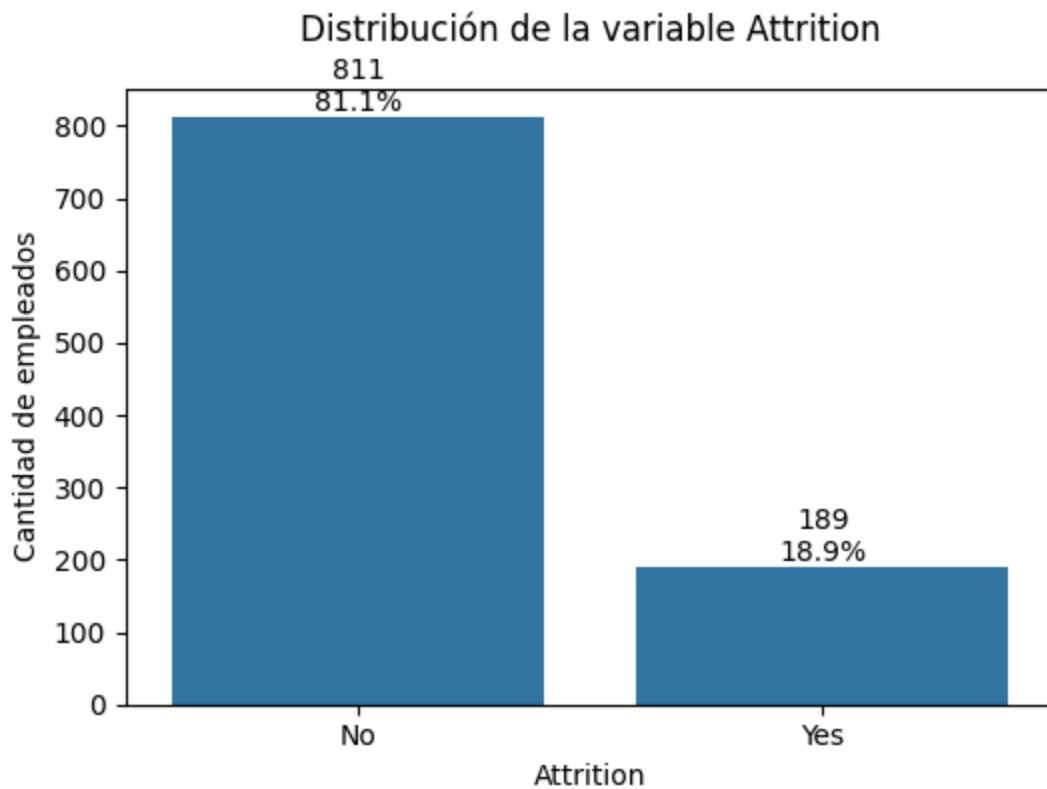
```
    ax.annotate(f'{cantidad}\n{porcentaje:.1f}%',  
               (p.get_x() + p.get_width() / 2, p.get_height()),  
               ha='center', va='bottom')
```

```
plt.title('Distribución de la variable Attrition', y=1.05)
```

```
plt.xlabel('Attrition')
```

```
plt.ylabel('Cantidad de empleados')
```

```
plt.show()
```



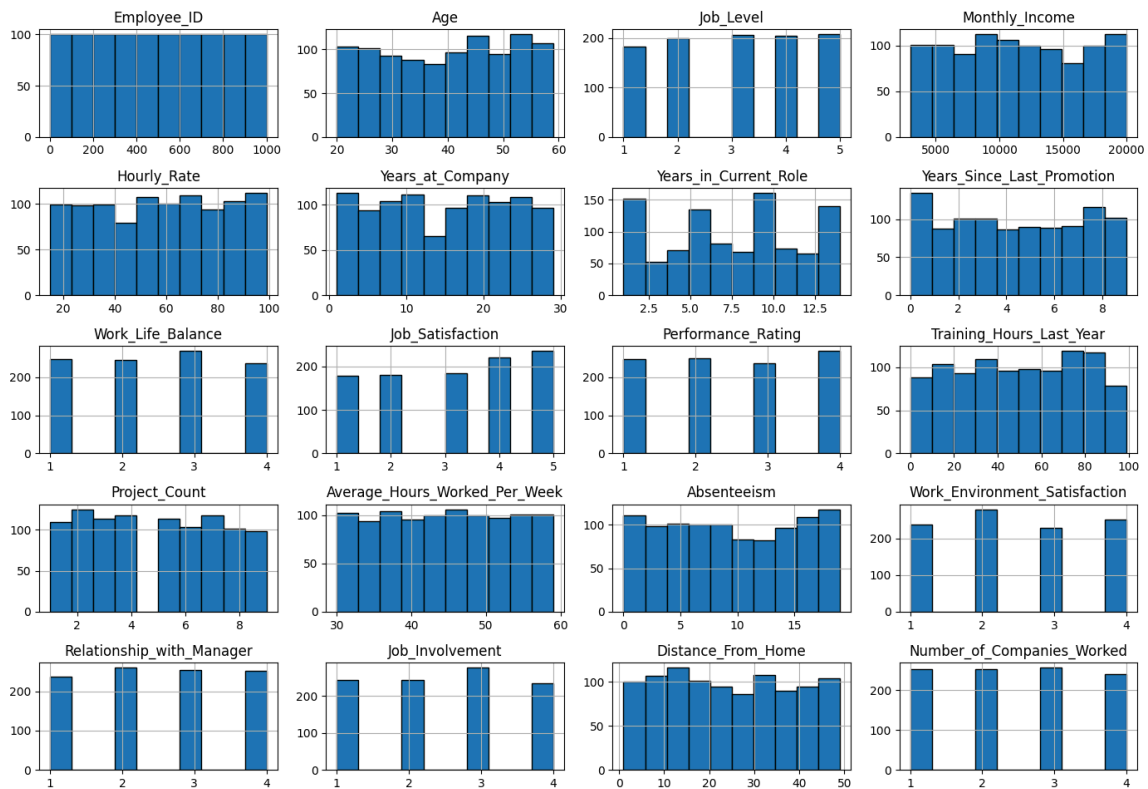
2. Análisis Exploratorio de Datos (EDA)

Histogramas de las variables numéricas

```
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
plt.figure()
df[numeric_cols].hist(figsize=(14, 10), edgecolor='black')
plt.suptitle('Distribución de variables numéricas', y=1.02)
plt.tight_layout()
plt.show()
```

<Figure size 640x480 with 0 Axes>

Distribución de variables numéricas



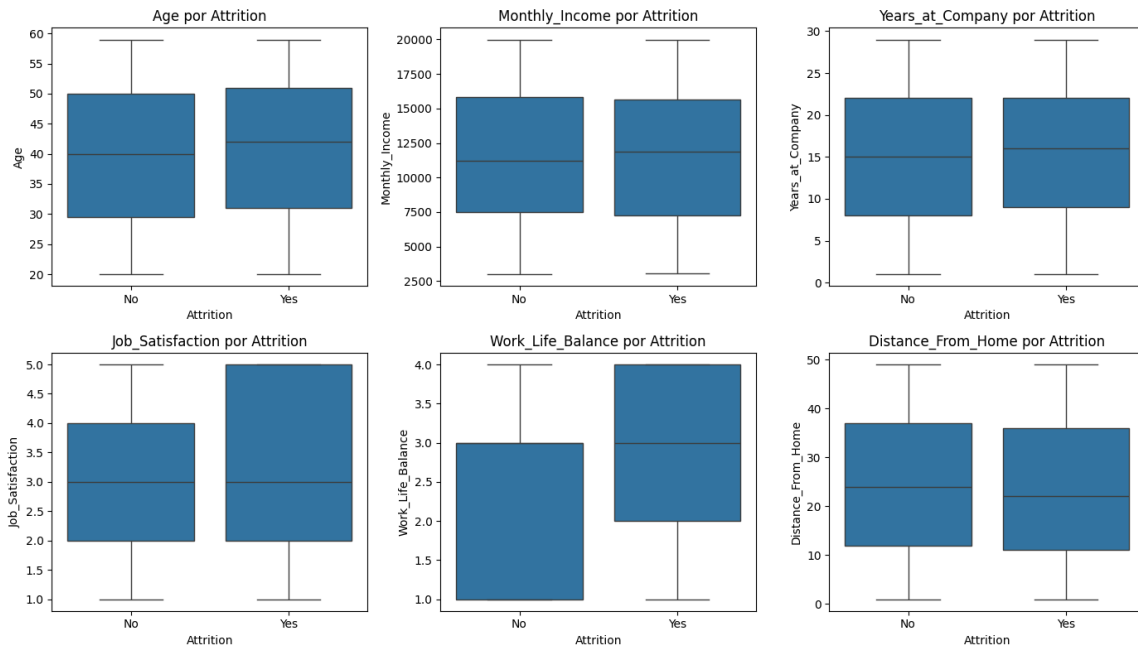
Boxplots de variables clave según Attrition

```
fig, axes = plt.subplots(2, 3, figsize=(14, 8))
```

```
vars_to_plot = ['Age', 'Monthly_Income', 'Years_at_Company',
                'Job_Satisfaction', 'Work_Life_Balance',
                'Distance_From_Home']
```

```
for ax, var in zip(axes.flatten(), vars_to_plot):
    sns.boxplot(x='Attrition', y=var, data=df, ax=ax)
    ax.set_title(f'{var} por Attrition')
```

```
plt.tight_layout()
plt.show()
```



Attrition por variables categóricas

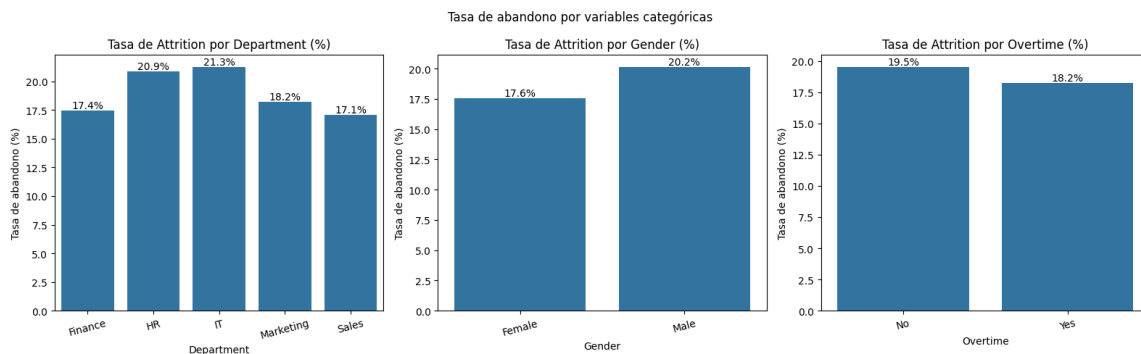
```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
```

```
for ax, col in zip(axes, ['Department', 'Gender', 'Overtime']):
    attrition_rate = df.groupby(col)['Attrition'].apply(
        lambda x: (x == 'Yes').mean() * 100
    ).reset_index()
    attrition_rate.columns = [col, 'Attrition_Rate']
    sns.barplot(x=col, y='Attrition_Rate', data=attrition_rate, ax=ax)
    for p in ax.patches:
        porcentaje = p.get_height()
        ax.annotate(f'{porcentaje:.1f}%',
                    (p.get_x() + p.get_width() / 2, p.get_height()),
                    ha='center', va='bottom')
    ax.set_title(f'Tasa de Attrition por {col} (%)')
    ax.set_ylabel('Tasa de abandono (%)')
    ax.tick_params(axis='x', rotation=15)
```

```
fig.suptitle('Tasa de abandono por variables categóricas')
```

```
plt.tight_layout()
```

```
plt.show()
```



3. Preprocesamiento de Datos

Eliminar columna identificadora (no aporta información predictiva)

```
df_model = df.drop(columns=['Employee_ID'])
```

Codificar variables categóricas con LabelEncoder

```
le = LabelEncoder()
```

```
categorical_cols = df_model.select_dtypes(include=
```

```
    ['object']).columns.tolist()
```

```
print("Variables categóricas a codificar:", categorical_cols)
```

```
for col in categorical_cols:
```

```
    df_model[col] = le.fit_transform(df_model[col])
```

```
df_model.head()
```

Variables categóricas a codificar: ['Gender', 'Marital_Status', 'Department', 'Job_Role', 'Overtime', 'Attrition']

	Age	Gender	Marital_Status	Department	Job_Role	Job_Level	Monthly_Income
0	58	0	1	2	3	1	15488
1	48	0	1	4	1	5	13079
2	34	1	1	3	1	1	13744
3	27	0	0	3	3	1	6809
4	40	1	0	3	2	1	10206

5 rows × 25 columns

Separar variables predictoras (X) y variable objetivo (y)

```
X = df_model.drop(columns=['Attrition'])
```

```
y = df_model['Attrition']
```

Dividir en conjunto de entrenamiento (80%) y prueba (20%)

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.20, random_state=2026
```

```
)
```

```
print(f"Tamaño entrenamiento: {X_train.shape}")
```

```
print(f"Tamaño prueba: {X_test.shape}")
```

Tamaño entrenamiento: (800, 24)

Tamaño prueba: (200, 24)

Escalar las variables (necesario para Regresión Logística y KNN)

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

4. Regresión Logística

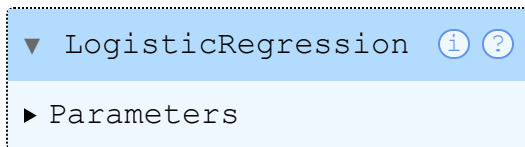
La **Regresión Logística** es un modelo de clasificación que estima la probabilidad de que una observación pertenezca a una clase determinada. Utiliza la función sigmoide para mapear valores reales al intervalo $[0, 1]$.

Se evaluarán tres configuraciones distintas:

- a) Parámetros por defecto
- b) Regularización L1 (lasso)
- c) Mayor número de iteraciones con penalización L2

a) Regresión Logística con parámetros por defecto

```
# Crear el objeto de clase Regresión Logística con parámetros por defecto
lr_a = LogisticRegression(random_state=2026)
# Ajustar el modelo
lr_a.fit(X_train_scaled, y_train)
```



```
# Obtener predicciones
y_pred_lr_a = lr_a.predict(X_test_scaled)

# Accuracy Score
accuracy_lr_a = accuracy_score(y_test, y_pred_lr_a)
print("El accuracy score que se tiene es %s" % accuracy_lr_a)
```

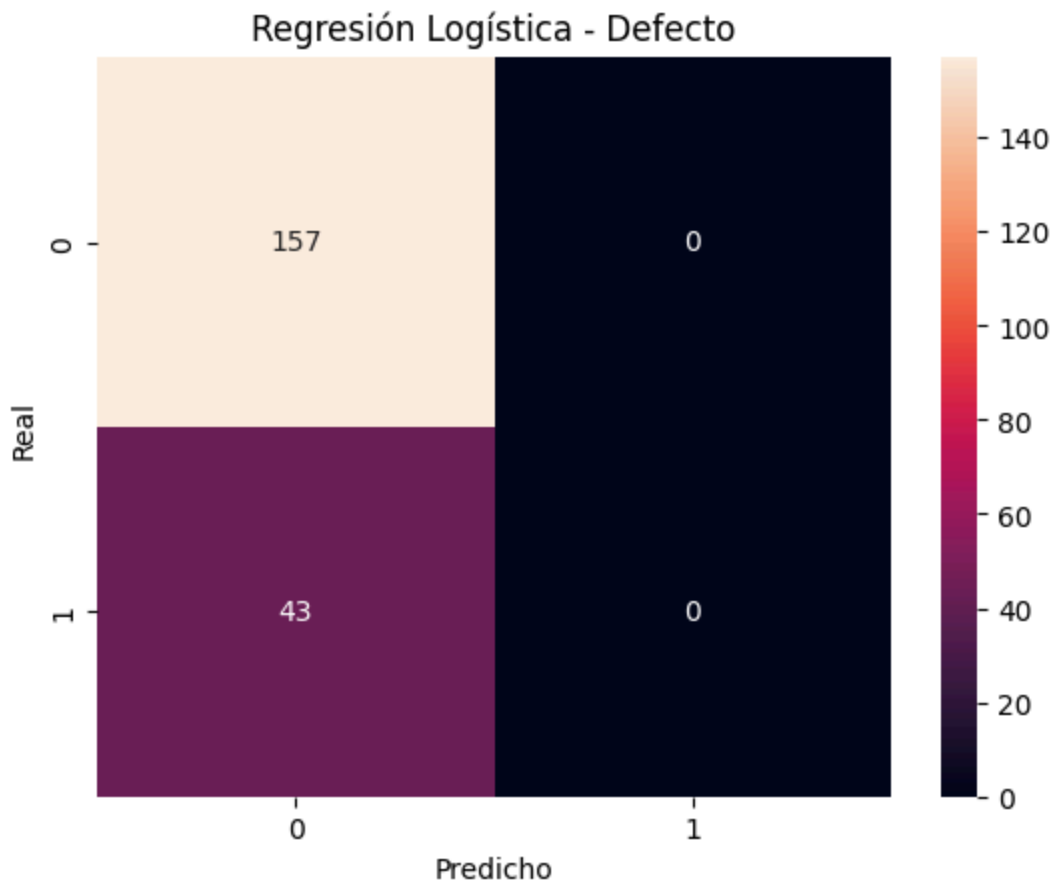
El accuracy score que se tiene es 0.785

```
from sklearn.metrics import confusion_matrix
```

```
# Obtener la matriz de confusión
cm = confusion_matrix(y_test, y_pred_lr_a)
sns.heatmap(cm, annot=True, fmt="d")

plt.title("Regresión Logística - Defecto")
plt.xlabel("Predicho")
plt.ylabel("Real")
```

```
Text(50.72222222222214, 0.5, 'Real')
```



Obtener la precisión, recall y f1

```
precision_lr_a = precision_score(y_test, y_pred_lr_a, average="macro")
```

```
recall_lr_a = recall_score(y_test, y_pred_lr_a, average="macro")
```

```
f1_lr_a = f1_score(y_test, y_pred_lr_a, average="macro")
```

```
metricas = pd.DataFrame({"Métricas": ["Precisión", "Recall", "F1"],
                          "Valores": [precision_lr_a, recall_lr_a, f1_lr_a]})
```

metricas

	Métricas	Valores
0	Precisión	0.392500
1	Recall	0.500000
2	F1	0.439776

b) Regresión Logística con regularización L1 (Lasso)

La regularización L1 produce modelos más dispersos al forzar algunos coeficientes a cero, actuando como un selector automático de variables.

Crear el objeto de clase Regresión Logística con regularización L1

```
lr_b = LogisticRegression(penalty='l1', solver='liblinear',
                          random_state=2026)
```

Ajustar el modelo

```
lr_b.fit(X_train_scaled, y_train)
```

▼ LogisticRegression ⓘ ?

► Parameters

Obtener predicciones

```
y_pred_lr_b = lr_b.predict(X_test_scaled)
```

Accuracy Score

```
accuracy_lr_b = accuracy_score(y_test, y_pred_lr_b)
```

```
print("El accuracy score que se tiene es %s" % accuracy_lr_b)
```

El accuracy score que se tiene es 0.785

Obtener La matriz de confusión

```
cm = confusion_matrix(y_test, y_pred_lr_b)
```

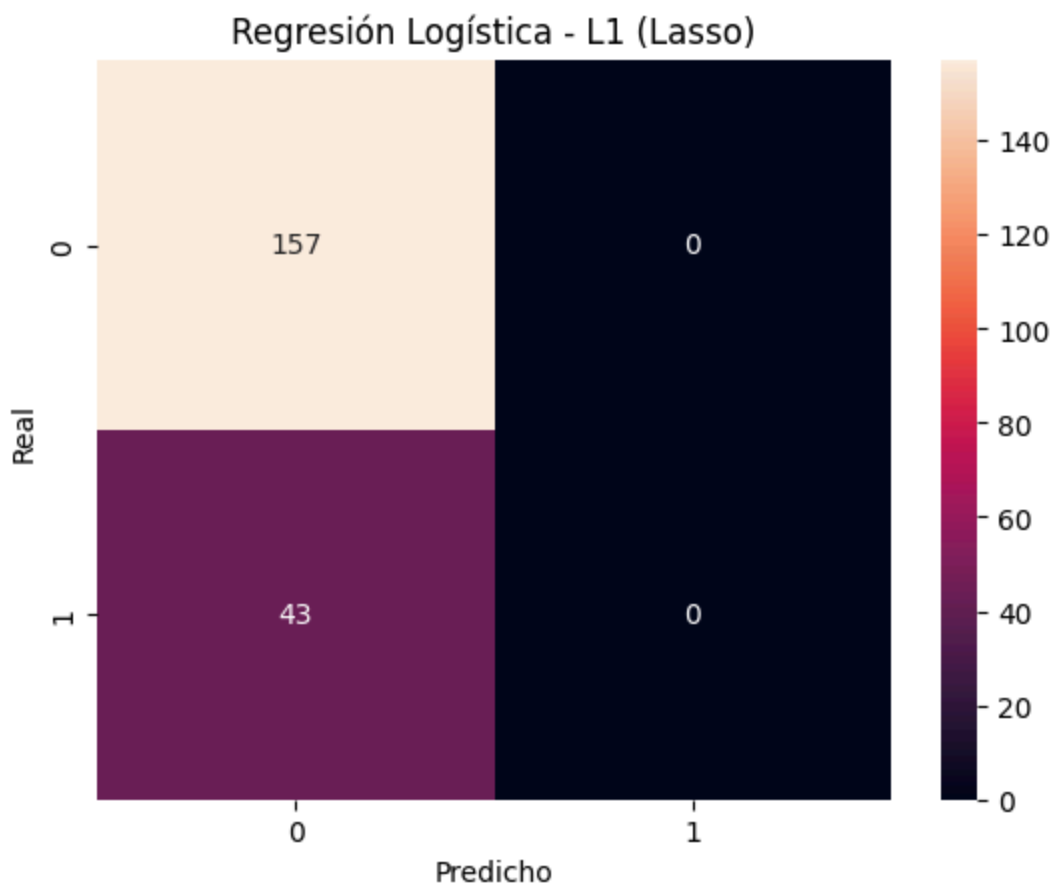
```
sns.heatmap(cm, annot=True, fmt="d")
```

```
plt.title("Regresión Logística - L1 (Lasso)")
```

```
plt.xlabel("Predicho")
```

```
plt.ylabel("Real")
```

```
Text(50.72222222222214, 0.5, 'Real')
```



Obtener la precisión, recall y f1

```
precision_lr_b = precision_score(y_test, y_pred_lr_b, average="macro")
```

```
recall_lr_b = recall_score(y_test, y_pred_lr_b, average="macro")
```

```
f1_lr_b = f1_score(y_test, y_pred_lr_b, average="macro")
```

```
metricas = pd.DataFrame({"Métricas": ["Precisión", "Recall", "F1"],
                        "Valores": [precision_lr_b, recall_lr_b, f1_lr_b]})

metricas

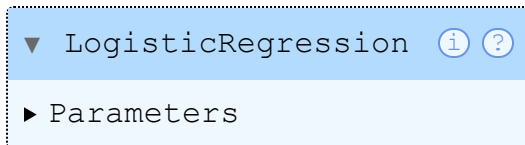
#dfi.export(metricas, "metricas_arbol_max_depth_5.png")
```

	Métricas	Valores	
0	Precisión	0.392500	
1	Recall	0.500000	
2	F1	0.439776	

c) Regresión Logística con mayor número de iteraciones y C=0.5

Reducir el valor de C aumenta la regularización, lo que puede mejorar la generalización del modelo.

```
# Crear el objeto de clase Regresión Logística con C=0.5 y max_iter=500
lr_c = LogisticRegression(C=0.5, max_iter=500, random_state=2026)
# Ajustar el modelo
lr_c.fit(X_train_scaled, y_train)
```



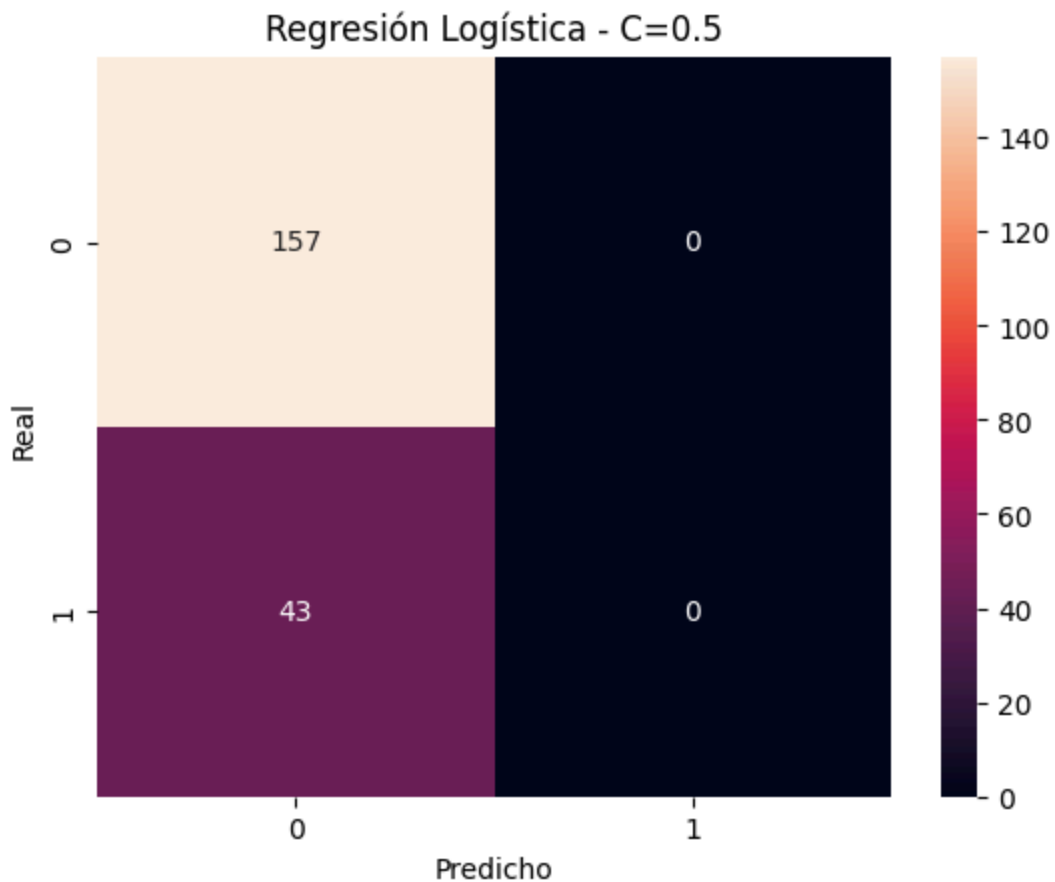
```
# Obtener predicciones
y_pred_lr_c = lr_c.predict(X_test_scaled)

# Accuracy Score
accuracy_lr_c = accuracy_score(y_test, y_pred_lr_c)
print("El accuracy score que se tiene es %s" % accuracy_lr_c)
```

El accuracy score que se tiene es 0.785

```
# Obtener la matriz de confusión
cm = confusion_matrix(y_test, y_pred_lr_c)
sns.heatmap(cm, annot=True, fmt="d")
plt.title("Regresión Logística - C=0.5")
plt.xlabel("Predicho")
plt.ylabel("Real")

Text(50.72222222222214, 0.5, 'Real')
```



Obtener la precisión, recall y f1

```
precision_lr_c = precision_score(y_test, y_pred_lr_c, average="macro")
recall_lr_c = recall_score(y_test, y_pred_lr_c, average="macro")
f1_lr_c = f1_score(y_test, y_pred_lr_c, average="macro")

metricas = pd.DataFrame({"Métricas": ["Precisión", "Recall", "F1"],
                          "Valores": [precision_lr_c, recall_lr_c, f1_lr_c]})

metricas
```

	Métricas	Valores
0	Precisión	0.392500
1	Recall	0.500000
2	F1	0.439776

Comparación de los modelos de Regresión Logística

A continuación se resumen las métricas de los tres modelos de Regresión Logística evaluados:

```
comparacion_lr = pd.DataFrame({
    'Modelo': ['RL - Defecto', 'RL - L1 (Lasso)', 'RL - C=0.5, iter=500'],
    'Accuracy': [accuracy_lr_a, accuracy_lr_b, accuracy_lr_c],
    'Precisión': [precision_lr_a, precision_lr_b, precision_lr_c],
    'Recall': [recall_lr_a, recall_lr_b, recall_lr_c],
})
```

```

        'F1': [f1_lr_a, f1_lr_b, f1_lr_c]
    })
    comparacion_lr

```

	Modelo	Accuracy	Precisión	Recall	F1
0	RL - Defecto	0.785	0.3925	0.5	0.439776
1	RL - L1 (Lasso)	0.785	0.3925	0.5	0.439776
2	RL - C=0.5, iter=500	0.785	0.3925	0.5	0.439776

5. K-Nearest Neighbors (KNN)

El algoritmo **KNN** clasifica una observación en función de la clase mayoritaria entre sus k vecinos más cercanos en el espacio de características.

Se evaluarán tres configuraciones:

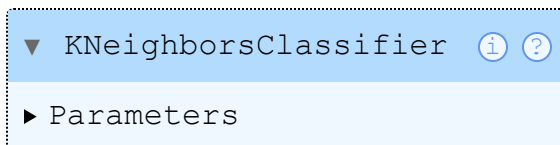
- a) k=5 (valor por defecto)
- b) k=3
- c) k=11

a) KNN con k=5 (parámetros por defecto)

```

# Crear el objeto de clase KNN con parámetros por defecto (k=5)
knn_a = KNeighborsClassifier()
# Ajustar el modelo
knn_a.fit(X_train_scaled, y_train)

```



```

# Obtener el accuracy score
y_pred_knn_a = knn_a.predict(X_test_scaled)
accuracy_knn_a = accuracy_score(y_test, y_pred_knn_a)
print("El accuracy score que se tiene es %s" % accuracy_knn_a)

```

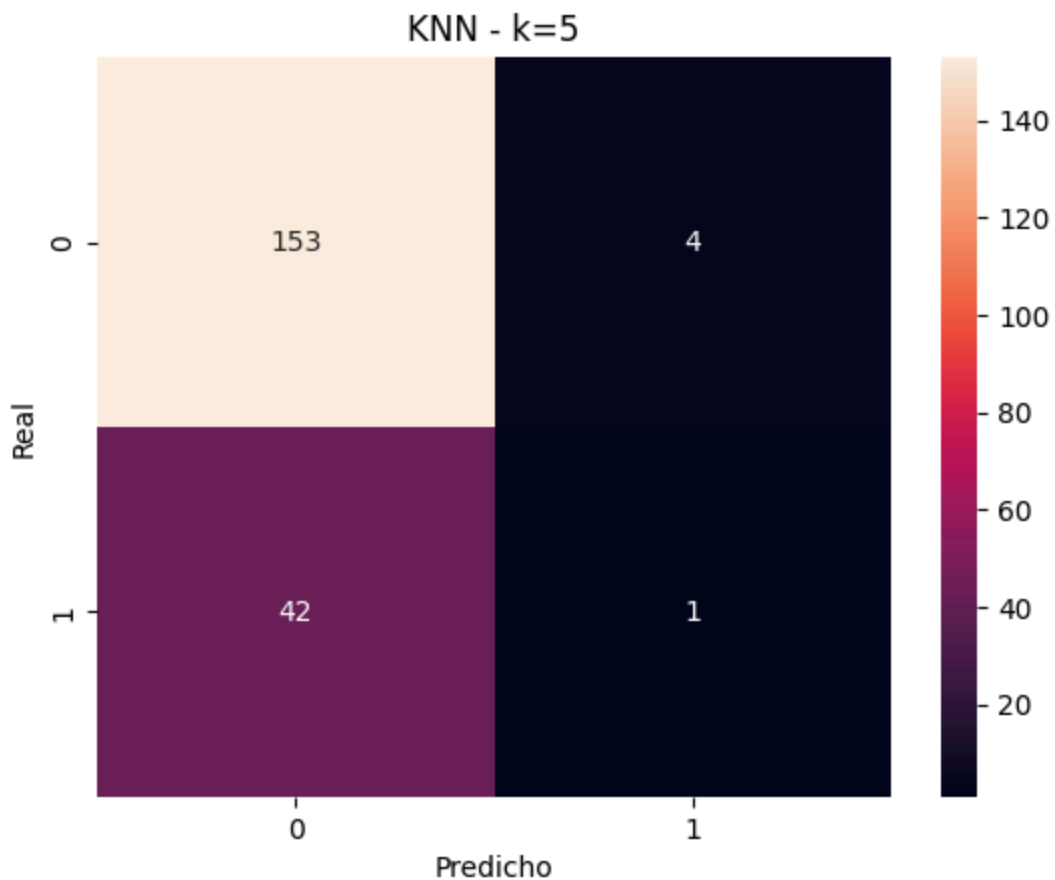
El accuracy score que se tiene es 0.77

```

# Obtener la matriz de confusión
cm = confusion_matrix(y_test, y_pred_knn_a)
sns.heatmap(cm, annot=True, fmt="d")
plt.title("KNN - k=5")
plt.xlabel("Predicho")
plt.ylabel("Real")

```

```
Text(50.72222222222214, 0.5, 'Real')
```



Obtener la precisión, recall y f1

```
precision_knn_a = precision_score(y_test, y_pred_knn_a, average="macro")
recall_knn_a = recall_score(y_test, y_pred_knn_a, average="macro")
f1_knn_a = f1_score(y_test, y_pred_knn_a, average="macro")
```

```
metricas = pd.DataFrame({"Métricas": ["Precisión", "Recall", "F1"],
                          "Valores": [precision_knn_a, recall_knn_a,
                                      f1_knn_a]})
```

metricas

	Métricas	Valores
0	Precisión	0.492308
1	Recall	0.498889
2	F1	0.455492

b) KNN con k=3

Con un valor de k más pequeño, el modelo es más sensible a las variaciones locales, lo que puede producir fronteras de decisión más irregulares.

Crear el objeto de clase KNN con k=3

```
knn_b = KNeighborsClassifier(n_neighbors=3)
```

Ajustar el modelo

```
knn_b.fit(X_train_scaled, y_train)
```

▼
KNeighborsClassifier
i
?

▶ Parameters

Obtener el accuracy score

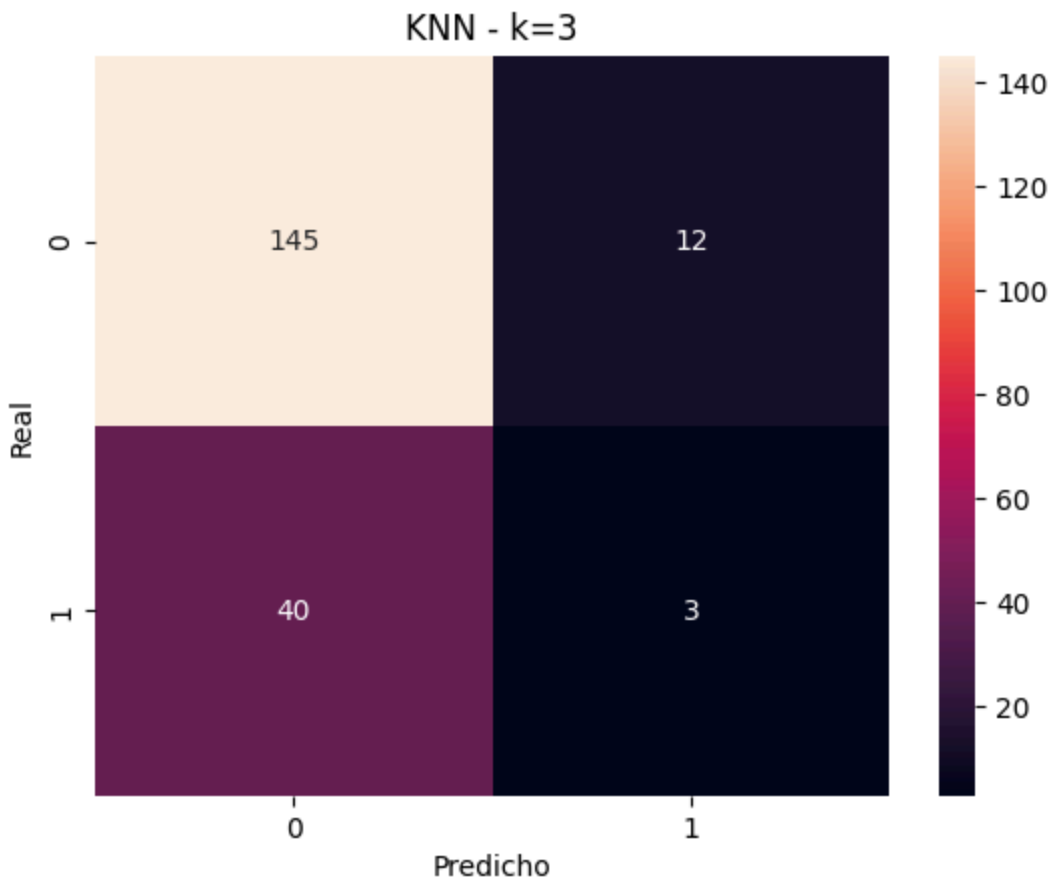
```
y_pred_knn_b = knn_b.predict(X_test_scaled)
accuracy_knn_b = accuracy_score(y_test, y_pred_knn_b)
print("El accuracy score que se tiene es %s" % accuracy_knn_b)
```

El accuracy score que se tiene es 0.74

Obtener la matriz de confusión

```
cm = confusion_matrix(y_test, y_pred_knn_b)
sns.heatmap(cm, annot=True, fmt="d")
plt.title("KNN - k=3")
plt.xlabel("Predicho")
plt.ylabel("Real")
```

```
Text(50.72222222222214, 0.5, 'Real')
```



Obtener la precisión, recall y f1

```
precision_knn_b = precision_score(y_test, y_pred_knn_b, average="macro")
recall_knn_b = recall_score(y_test, y_pred_knn_b, average="macro")
f1_knn_b = f1_score(y_test, y_pred_knn_b, average="macro")
```

```
metricas = pd.DataFrame({"Métricas": ["Precisión", "Recall", "F1"],
```

```

        "Valores": [precision_knn_b, recall_knn_b,
                    f1_knn_b]})
metricas

```

	Métricas	Valores
0	Precisión	0.491892
1	Recall	0.496667
2	F1	0.475701

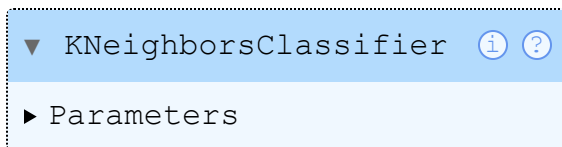
c) KNN con k=11

Con un valor de k más grande, el modelo suaviza las fronteras de decisión y puede ser más robusto al ruido en los datos.

```

# Crear el objeto de clase KNN con k=11
knn_c = KNeighborsClassifier(n_neighbors=11)
# Ajustar el modelo
knn_c.fit(X_train_scaled, y_train)

```



```

# Obtener el accuracy score
y_pred_knn_c = knn_c.predict(X_test_scaled)
accuracy_knn_c = accuracy_score(y_test, y_pred_knn_c)
print("El accuracy score que se tiene es %s" % accuracy_knn_c)

```

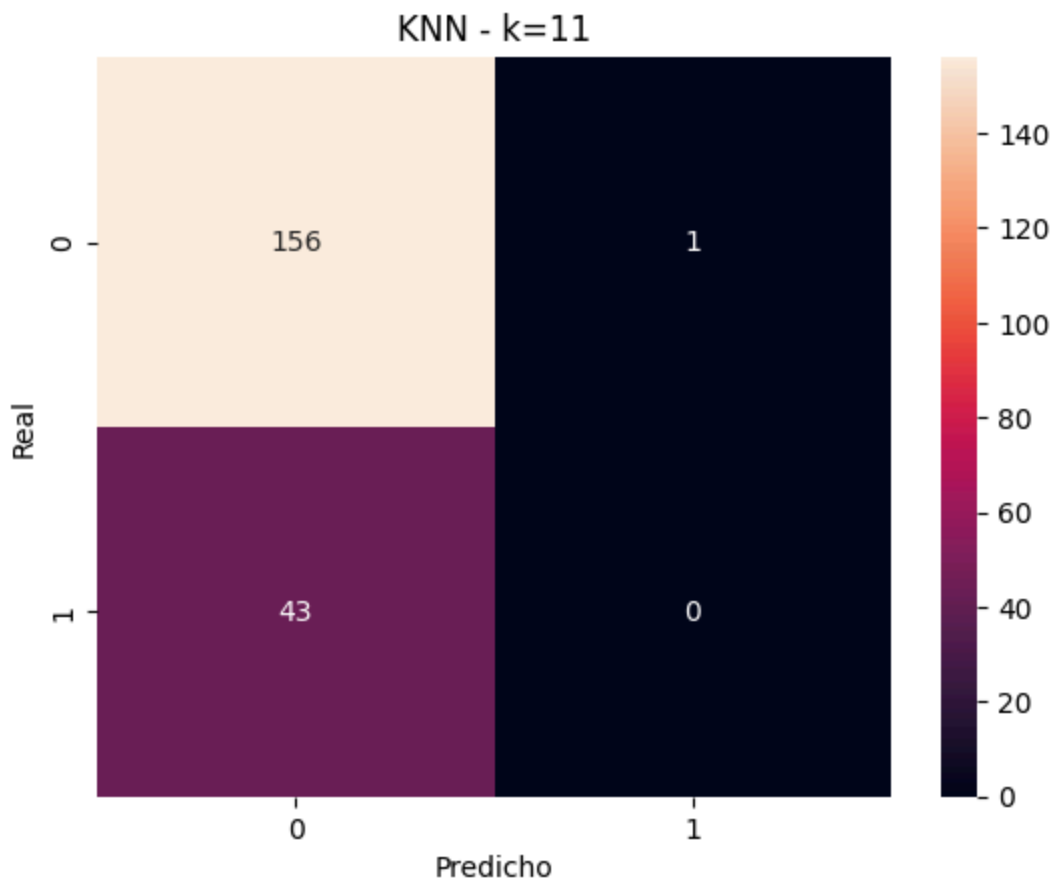
El accuracy score que se tiene es 0.78

```

# Obtener la matriz de confusión
cm = confusion_matrix(y_test, y_pred_knn_c)
sns.heatmap(cm, annot=True, fmt="d")
plt.title("KNN - k=11")
plt.xlabel("Predicho")
plt.ylabel("Real")

```

```
Text(50.72222222222214, 0.5, 'Real')
```



Obtener la precisión, recall y f1

```
precision_knn_c = precision_score(y_test, y_pred_knn_c, average="macro")
recall_knn_c = recall_score(y_test, y_pred_knn_c, average="macro")
f1_knn_c = f1_score(y_test, y_pred_knn_c, average="macro")
```

```
metricas = pd.DataFrame({"Métricas": ["Precisión", "Recall", "F1"],
                        "Valores": [precision_knn_c, recall_knn_c,
                                     f1_knn_c]})
```

metricas

	Métricas	Valores
0	Precisión	0.391960
1	Recall	0.496815
2	F1	0.438202

Comparación de los modelos KNN

```
comparacion_knn = pd.DataFrame({
    'Modelo': ['KNN - k=5 (defecto)', 'KNN - k=3', 'KNN - k=11'],
    'Accuracy': [accuracy_knn_a, accuracy_knn_b, accuracy_knn_c],
    'Precisión': [precision_knn_a, precision_knn_b, precision_knn_c],
    'Recall': [recall_knn_a, recall_knn_b, recall_knn_c],
    'F1': [f1_knn_a, f1_knn_b, f1_knn_c]
})
```

```
})  
comparacion_knn
```

	Modelo	Accuracy	Precisión	Recall	F1
0	KNN - k=5 (defecto)	0.77	0.492308	0.498889	0.455492
1	KNN - k=3	0.74	0.491892	0.496667	0.475701
2	KNN - k=11	0.78	0.391960	0.496815	0.438202

6.Árbol de Decisiones

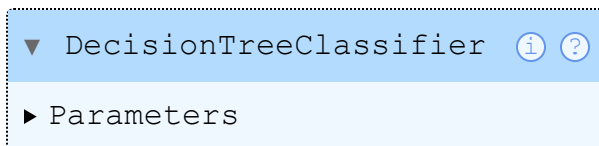
Un **árbol de decisión** consiste en una secuencia de condiciones sobre las distintas variables predictoras y su relación con la variable de respuesta. Estas condiciones se van sucediendo en distintos caminos del árbol, dividiendo el espacio de los predictores en diferentes regiones.

Se evaluarán tres configuraciones:

- a) Parámetros por defecto
- b) Profundidad máxima max_depth=5
- c) Mínimo de muestras en hoja min_samples_leaf=4

a) Árbol de Decisiones con parámetros por defecto

```
# Crear el objeto de clase árbol  
tree_clf_a = DecisionTreeClassifier(random_state=2026)  
# Ajustar el árbol  
tree_clf_a.fit(X_train, y_train)
```

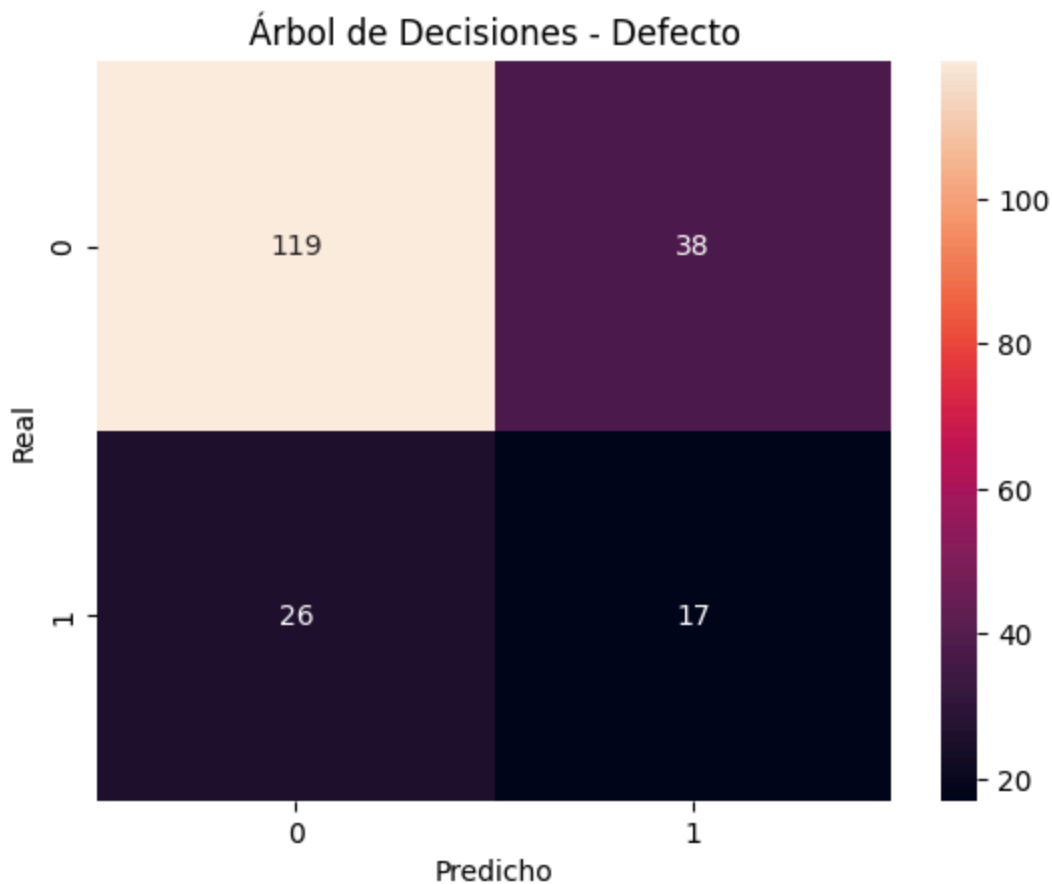


```
# Obtener el accuracy score  
y_pred_a = tree_clf_a.predict(X_test)  
accuracy_a = accuracy_score(y_test, y_pred_a)  
print("El accuracy score que se tiene es %s" % accuracy_a)
```

El accuracy score que se tiene es 0.68

```
# Obtener la matriz de confusión  
cm = confusion_matrix(y_test, y_pred_a)  
sns.heatmap(cm, annot=True, fmt="d")  
plt.title("Árbol de Decisiones - Defecto")  
plt.xlabel("Predicho")  
plt.ylabel("Real")
```

```
Text(50.72222222222214, 0.5, 'Real')
```



Obtener la precisión, recall y f1

```
precision_a = precision_score(y_test, y_pred_a, average="macro")
```

```
recall_a = recall_score(y_test, y_pred_a, average="macro")
```

```
f1_a = f1_score(y_test, y_pred_a, average="macro")
```

```
metricas = pd.DataFrame({"Métricas": ["Precisión", "Recall", "F1"],
                          "Valores": [precision_a, recall_a, f1_a]})
```

```
metricas
```

	Métricas	Valores
0	Precisión	0.564890
1	Recall	0.576655
2	F1	0.567509

b) Árbol de Decisiones con profundidad máxima `max_depth=5`

Una manera sencilla de evitar el posible sobreajuste del árbol es fijar un número de profundidad máxima, es decir, del número de divisiones sucesivas que se realizan.

Crear el objeto de clase árbol con `max_depth=5`

```
tree_clf_b = DecisionTreeClassifier(max_depth=5, random_state=2026)
```

Ajustar el árbol

```
tree_clf_b.fit(X_train, y_train)
```

```
plt.figure(figsize=(14,10))
```

<Figure size 1400x1000 with 0 Axes>

<Figure size 1400x1000 with 0 Axes>

Visualizar el árbol de decisión

tree.plot_tree(tree_clf_b, filled=True)

```
[Text(0.5703125, 0.9166666666666666, 'x[10] <= 6.5\ngini = 0.298\nsamples = 800\nvalue = [654, 146]'),
 Text(0.2864583333333333, 0.75, 'x[14] <= 56.5\ngini = 0.325\nsamples = 553\nvalue = [440, 113]'),
 Text(0.42838541666666663, 0.8333333333333333, 'True '),
 Text(0.15625, 0.5833333333333334, 'x[5] <= 3.5\ngini = 0.36\nsamples = 306\nvalue = [234.0, 72.0]'),
 Text(0.08333333333333333, 0.4166666666666667, 'x[18] <= 4.5\ngini = 0.408\nsamples = 175\nvalue = [125, 50]'),
 Text(0.041666666666666664, 0.25, 'x[12] <= 2.5\ngini = 0.236\nsamples = 44\nvalue = [38, 6]'),
 Text(0.020833333333333332, 0.08333333333333333, 'gini = 0.473\nsamples = 13\nvalue = [8, 5]'),
 Text(0.0625, 0.08333333333333333, 'gini = 0.062\nsamples = 31\nvalue = [30, 1]'),
 Text(0.125, 0.25, 'x[20] <= 2.5\ngini = 0.446\nsamples = 131\nvalue = [87, 44]'),
 Text(0.10416666666666667, 0.08333333333333333, 'gini = 0.375\nsamples = 68\nvalue = [51, 17]'),
 Text(0.14583333333333334, 0.08333333333333333, 'gini = 0.49\nsamples = 63\nvalue = [36, 27]'),
 Text(0.22916666666666666, 0.4166666666666667, 'x[6] <= 19878.5\ngini = 0.279\nsamples = 131\nvalue = [109, 22]'),
 Text(0.20833333333333334, 0.25, 'x[14] <= 52.5\ngini = 0.262\nsamples = 129\nvalue = [109, 20]'),
 Text(0.1875, 0.08333333333333333, 'gini = 0.225\nsamples = 116\nvalue = [101.0, 15.0]'),
 Text(0.22916666666666666, 0.08333333333333333, 'gini = 0.473\nsamples = 13\nvalue = [8, 5]'),
 Text(0.25, 0.25, 'gini = 0.0\nsamples = 2\nvalue = [0, 2]'),
 Text(0.4166666666666667, 0.5833333333333334, 'x[8] <= 6.5\ngini = 0.277\nsamples = 247\nvalue = [206, 41]'),
 Text(0.3333333333333333, 0.4166666666666667, 'x[8] <= 2.5\ngini = 0.44\nsamples = 52\nvalue = [35, 17]'),
 Text(0.2916666666666667, 0.25, 'x[10] <= 4.5\ngini = 0.198\nsamples = 18\nvalue = [16, 2]'),
 Text(0.27083333333333333, 0.08333333333333333, 'gini = 0.0\nsamples = 13\nvalue = [13, 0]'),
 Text(0.3125, 0.08333333333333333, 'gini = 0.48\nsamples = 5\nvalue = [3, 2]'),
 Text(0.375, 0.25, 'x[9] <= 6.5\ngini = 0.493\nsamples = 34\nvalue = [19, 15]'),
 Text(0.3541666666666667, 0.08333333333333333, 'gini = 0.337\nsamples = 14\nvalue = [11, 3]'),
 Text(0.39583333333333333, 0.08333333333333333, 'gini = 0.48\nsamples = 20\nvalue = [8, 12]'),
 Text(0.5, 0.4166666666666667, 'x[7] <= 26.5\ngini = 0.216\nsamples =
```

```

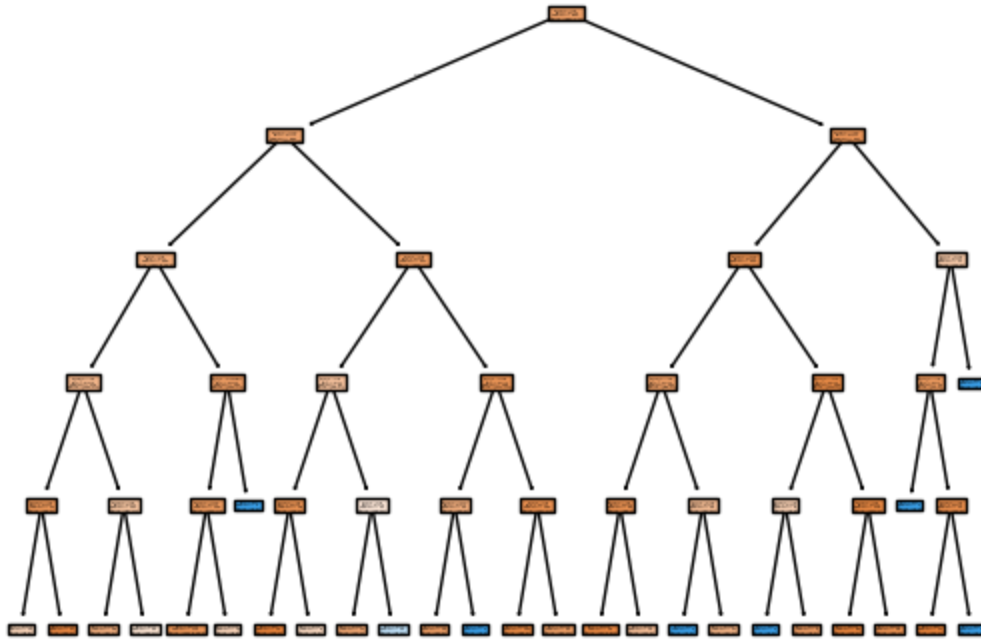
195\nvalue = [171, 24]'),
  Text(0.4583333333333333, 0.25, 'x[14] <= 91.0\ngini = 0.375\nsamples =
32\nvalue = [24, 8]'),
  Text(0.4375, 0.0833333333333333, 'gini = 0.285\nsamples = 29\nvalue = [24,
5]'),
  Text(0.4791666666666667, 0.0833333333333333, 'gini = 0.0\nsamples =
3\nvalue = [0, 3]'),
  Text(0.5416666666666666, 0.25, 'x[5] <= 3.5\ngini = 0.177\nsamples =
163\nvalue = [147, 16]'),
  Text(0.5208333333333334, 0.0833333333333333, 'gini = 0.095\nsamples =
100\nvalue = [95, 5]'),
  Text(0.5625, 0.0833333333333333, 'gini = 0.288\nsamples = 63\nvalue = [52,
11]'),
  Text(0.8541666666666666, 0.75, 'x[18] <= 18.5\ngini = 0.232\nsamples =
247\nvalue = [214, 33]'),
  Text(0.7122395833333333, 0.8333333333333333, ' False'),
  Text(0.75, 0.5833333333333334, 'x[22] <= 21.5\ngini = 0.207\nsamples =
230\nvalue = [203, 27]'),
  Text(0.6666666666666666, 0.4166666666666667, 'x[12] <= 3.5\ngini =
0.299\nsamples = 104\nvalue = [85, 19]'),
  Text(0.625, 0.25, 'x[16] <= 7.5\ngini = 0.192\nsamples = 65\nvalue = [58,
7]'),
  Text(0.6041666666666666, 0.0833333333333333, 'gini = 0.078\nsamples =
49\nvalue = [47.0, 2.0]'),
  Text(0.6458333333333334, 0.0833333333333333, 'gini = 0.43\nsamples =
16\nvalue = [11, 5]'),
  Text(0.7083333333333334, 0.25, 'x[7] <= 17.5\ngini = 0.426\nsamples =
39\nvalue = [27, 12]'),
  Text(0.6875, 0.0833333333333333, 'gini = 0.0\nsamples = 2\nvalue = [0,
2]'),
  Text(0.7291666666666666, 0.0833333333333333, 'gini = 0.394\nsamples =
37\nvalue = [27, 10]'),
  Text(0.8333333333333334, 0.4166666666666667, 'x[17] <= 31.5\ngini =
0.119\nsamples = 126\nvalue = [118, 8]'),
  Text(0.7916666666666666, 0.25, 'x[13] <= 1.5\ngini = 0.469\nsamples =
8\nvalue = [5, 3]'),
  Text(0.7708333333333334, 0.0833333333333333, 'gini = 0.0\nsamples =
2\nvalue = [0, 2]'),
  Text(0.8125, 0.0833333333333333, 'gini = 0.278\nsamples = 6\nvalue = [5,
1]'),
  Text(0.875, 0.25, 'x[10] <= 8.5\ngini = 0.081\nsamples = 118\nvalue = [113,
5]'),
  Text(0.8541666666666666, 0.0833333333333333, 'gini = 0.025\nsamples =
80\nvalue = [79, 1]'),
  Text(0.8958333333333334, 0.0833333333333333, 'gini = 0.188\nsamples =
38\nvalue = [34, 4]'),
  Text(0.9583333333333334, 0.5833333333333334, 'x[7] <= 80.5\ngini =
0.457\nsamples = 17\nvalue = [11, 6]'),
  Text(0.9375, 0.4166666666666667, 'x[11] <= 1.5\ngini = 0.26\nsamples =
13\nvalue = [11, 2]'),
  Text(0.9166666666666666, 0.25, 'gini = 0.0\nsamples = 1\nvalue = [0, 1]'),
  Text(0.9583333333333334, 0.25, 'x[19] <= 3.5\ngini = 0.153\nsamples =
12\nvalue = [11, 1]'),
  Text(0.9375, 0.0833333333333333, 'gini = 0.0\nsamples = 11\nvalue = [11,

```

```

0]'),
  Text(0.9791666666666666, 0.08333333333333333, 'gini = 0.0\nsamples =
1\nvalue = [0, 1]'),
  Text(0.9791666666666666, 0.4166666666666667, 'gini = 0.0\nsamples = 4\nvalue
= [0, 4]')]

```



```

# Obtener el accuracy score
y_pred_b = tree_clf_b.predict(X_test)
accuracy_b = accuracy_score(y_test, y_pred_b)
print("El accuracy score que se tiene es %s" % accuracy_b)

```

El accuracy score que se tiene es 0.765

```

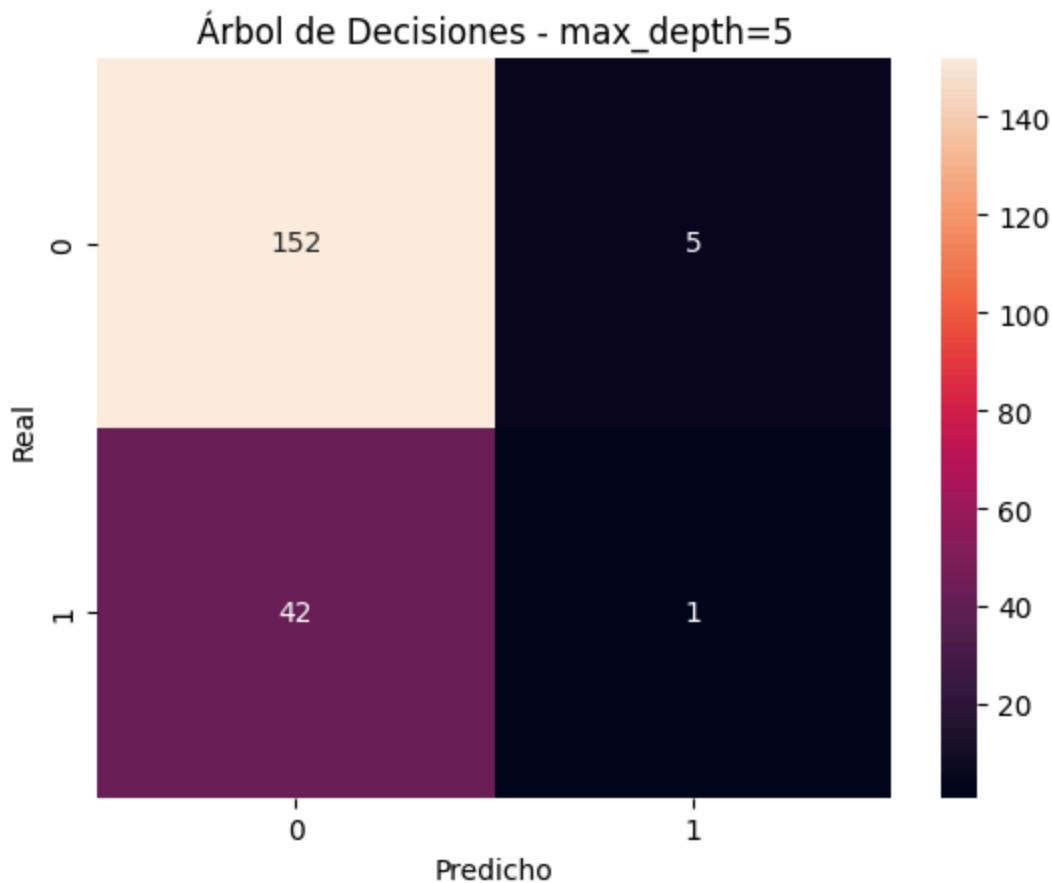
# Obtener la matriz de confusión
cm = confusion_matrix(y_test, y_pred_b)
sns.heatmap(cm, annot=True, fmt="d")
plt.title("Árbol de Decisiones - max_depth=5")
plt.xlabel("Predicho")
plt.ylabel("Real")

```

```

Text(50.72222222222214, 0.5, 'Real')

```



Obtener la precisión, recall y f1

```
precision_b = precision_score(y_test, y_pred_b, average="macro")
```

```
recall_b = recall_score(y_test, y_pred_b, average="macro")
```

```
f1_b = f1_score(y_test, y_pred_b, average="macro")
```

```
metricas = pd.DataFrame({"Métricas": ["Precisión", "Recall", "F1"],
                          "Valores": [precision_b, recall_b, f1_b]})
```

```
metricas
```

	Métricas	Valores
0	Precisión	0.475086
1	Recall	0.495704
2	F1	0.453457

c) Árbol de Decisiones indicando que el mínimo número de observaciones que debe contener cada nodo hoja es de 4.

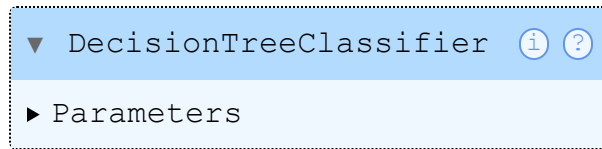
Otra manera de restringir la longitud del árbol de decisión sin fijar la profundidad máxima es indicando el número mínimo de observaciones que debe contener cada nodo hoja. En este caso se fijará el valor de min_samples_leaf=4.

Crear el objeto de clase árbol

```
tree_clf_c = DecisionTreeClassifier(min_samples_leaf=4, random_state=2026) #  
4 muestras
```

Ajustar el árbol

```
tree_clf_c.fit(X_train, y_train)
```



```
from sklearn import tree
```

```
tree.plot_tree(tree_clf_c, filled=True)
```

```
[Text(0.6492718446601942, 0.9583333333333334, 'x[10] <= 6.5\ngini =  
0.298\nsamples = 800\nvalue = [654, 146]'),  
Text(0.375, 0.875, 'x[14] <= 56.5\ngini = 0.325\nsamples = 553\nvalue =  
[440, 113]'),  
Text(0.5121359223300971, 0.9166666666666667, 'True '),  
Text(0.2275485436893204, 0.7916666666666666, 'x[5] <= 3.5\ngini =  
0.36\nsamples = 306\nvalue = [234.0, 72.0]'),  
Text(0.1104368932038835, 0.7083333333333334, 'x[18] <= 4.5\ngini =  
0.408\nsamples = 175\nvalue = [125, 50]'),  
Text(0.038834951456310676, 0.625, 'x[12] <= 2.5\ngini = 0.236\nsamples =  
44\nvalue = [38, 6]'),  
Text(0.019417475728155338, 0.5416666666666666, 'x[8] <= 15.5\ngini =  
0.473\nsamples = 13\nvalue = [8, 5]'),  
Text(0.009708737864077669, 0.4583333333333333, 'gini = 0.32\nsamples =  
5\nvalue = [1, 4]'),  
Text(0.02912621359223301, 0.4583333333333333, 'x[9] <= 6.5\ngini =  
0.219\nsamples = 8\nvalue = [7, 1]'),  
Text(0.019417475728155338, 0.375, 'gini = 0.375\nsamples = 4\nvalue = [3,  
1]'),  
Text(0.038834951456310676, 0.375, 'gini = 0.0\nsamples = 4\nvalue = [4,  
0]'),  
Text(0.05825242718446602, 0.5416666666666666, 'x[7] <= 28.0\ngini =  
0.062\nsamples = 31\nvalue = [30, 1]'),  
Text(0.04854368932038835, 0.4583333333333333, 'gini = 0.375\nsamples =  
4\nvalue = [3, 1]'),  
Text(0.06796116504854369, 0.4583333333333333, 'gini = 0.0\nsamples =  
27\nvalue = [27, 0]'),  
Text(0.1820388349514563, 0.625, 'x[20] <= 2.5\ngini = 0.446\nsamples =  
131\nvalue = [87, 44]'),  
Text(0.11165048543689321, 0.5416666666666666, 'x[8] <= 13.5\ngini =  
0.375\nsamples = 68\nvalue = [51, 17]'),  
Text(0.08737864077669903, 0.4583333333333333, 'x[8] <= 6.5\ngini =  
0.137\nsamples = 27\nvalue = [25, 2]'),  
Text(0.07766990291262135, 0.375, 'x[21] <= 2.5\ngini = 0.26\nsamples =  
13\nvalue = [11, 2]'),  
Text(0.06796116504854369, 0.2916666666666667, 'gini = 0.0\nsamples =  
8\nvalue = [8, 0]'),  
Text(0.08737864077669903, 0.2916666666666667, 'gini = 0.48\nsamples =  
5\nvalue = [3, 2]'),  
Text(0.0970873786407767, 0.375, 'gini = 0.0\nsamples = 14\nvalue = [14,
```

```
0]'),
  Text(0.13592233009708737, 0.4583333333333333, 'x[16] <= 5.5\ngini =
0.464\nsamples = 41\nvalue = [26, 15]'),
  Text(0.11650485436893204, 0.375, 'x[6] <= 7027.5\ngini = 0.49\nsamples =
21\nvalue = [9, 12]'),
  Text(0.10679611650485436, 0.2916666666666667, 'gini = 0.245\nsamples =
7\nvalue = [6, 1]'),
  Text(0.1262135922330097, 0.2916666666666667, 'x[14] <= 9.0\ngini =
0.337\nsamples = 14\nvalue = [3, 11]'),
  Text(0.11650485436893204, 0.20833333333333334, 'gini = 0.375\nsamples =
4\nvalue = [3, 1]'),
  Text(0.13592233009708737, 0.20833333333333334, 'gini = 0.0\nsamples =
10\nvalue = [0, 10]'),
  Text(0.1553398058252427, 0.375, 'x[6] <= 8283.0\ngini = 0.255\nsamples =
20\nvalue = [17, 3]'),
  Text(0.14563106796116504, 0.2916666666666667, 'gini = 0.49\nsamples =
7\nvalue = [4, 3]'),
  Text(0.1650485436893204, 0.2916666666666667, 'gini = 0.0\nsamples =
13\nvalue = [13, 0]'),
  Text(0.2524271844660194, 0.5416666666666666, 'x[3] <= 3.5\ngini =
0.49\nsamples = 63\nvalue = [36, 27]'),
  Text(0.24271844660194175, 0.4583333333333333, 'x[16] <= 5.5\ngini =
0.469\nsamples = 56\nvalue = [35, 21]'),
  Text(0.20388349514563106, 0.375, 'x[15] <= 0.5\ngini = 0.383\nsamples =
31\nvalue = [23, 8]'),
  Text(0.18446601941747573, 0.2916666666666667, 'x[8] <= 6.5\ngini =
0.5\nsamples = 14\nvalue = [7, 7]'),
  Text(0.17475728155339806, 0.20833333333333334, 'gini = 0.32\nsamples =
5\nvalue = [1, 4]'),
  Text(0.1941747572815534, 0.20833333333333334, 'x[6] <= 8181.5\ngini =
0.444\nsamples = 9\nvalue = [6, 3]'),
  Text(0.18446601941747573, 0.125, 'gini = 0.48\nsamples = 5\nvalue = [2,
3]'),
  Text(0.20388349514563106, 0.125, 'gini = 0.0\nsamples = 4\nvalue = [4, 0]'),
  Text(0.22330097087378642, 0.2916666666666667, 'x[12] <= 1.5\ngini =
0.111\nsamples = 17\nvalue = [16, 1]'),
  Text(0.21359223300970873, 0.20833333333333334, 'gini = 0.375\nsamples =
4\nvalue = [3, 1]'),
  Text(0.23300970873786409, 0.20833333333333334, 'gini = 0.0\nsamples =
13\nvalue = [13, 0]'),
  Text(0.2815533980582524, 0.375, 'x[19] <= 2.5\ngini = 0.499\nsamples =
25\nvalue = [12, 13]'),
  Text(0.2621359223300971, 0.2916666666666667, 'x[0] <= 36.0\ngini =
0.355\nsamples = 13\nvalue = [3, 10]'),
  Text(0.2524271844660194, 0.20833333333333334, 'gini = 0.5\nsamples =
6\nvalue = [3, 3]'),
  Text(0.27184466019417475, 0.20833333333333334, 'gini = 0.0\nsamples =
7\nvalue = [0, 7]'),
  Text(0.30097087378640774, 0.2916666666666667, 'x[6] <= 13767.5\ngini =
0.375\nsamples = 12\nvalue = [9, 3]'),
  Text(0.2912621359223301, 0.20833333333333334, 'gini = 0.5\nsamples =
6\nvalue = [3, 3]'),
  Text(0.3106796116504854, 0.20833333333333334, 'gini = 0.0\nsamples =
6\nvalue = [6, 0]'),
```

```
Text(0.2621359223300971, 0.4583333333333333, 'gini = 0.245\nsamples = 7\nvalue = [1, 6]'),
Text(0.3446601941747573, 0.7083333333333334, 'x[12] <= 1.5\ngini = 0.279\nsamples = 131\nvalue = [109, 22]'),
Text(0.32038834951456313, 0.625, 'x[20] <= 2.5\ngini = 0.465\nsamples = 19\nvalue = [12, 7]'),
Text(0.3106796116504854, 0.5416666666666666, 'x[8] <= 12.0\ngini = 0.486\nsamples = 12\nvalue = [5, 7]'),
Text(0.30097087378640774, 0.4583333333333333, 'gini = 0.444\nsamples = 6\nvalue = [4, 2]'),
Text(0.32038834951456313, 0.4583333333333333, 'gini = 0.278\nsamples = 6\nvalue = [1, 5]'),
Text(0.3300970873786408, 0.5416666666666666, 'gini = 0.0\nsamples = 7\nvalue = [7, 0]'),
Text(0.36893203883495146, 0.625, 'x[23] <= 2.5\ngini = 0.232\nsamples = 112\nvalue = [97, 15]'),
Text(0.34951456310679613, 0.5416666666666666, 'x[7] <= 94.5\ngini = 0.32\nsamples = 60\nvalue = [48, 12]'),
Text(0.33980582524271846, 0.4583333333333333, 'x[8] <= 26.5\ngini = 0.274\nsamples = 55\nvalue = [46, 9]'),
Text(0.3300970873786408, 0.375, 'x[22] <= 4.5\ngini = 0.215\nsamples = 49\nvalue = [43, 6]'),
Text(0.32038834951456313, 0.2916666666666667, 'gini = 0.48\nsamples = 5\nvalue = [3, 2]'),
Text(0.33980582524271846, 0.2916666666666667, 'x[22] <= 26.5\ngini = 0.165\nsamples = 44\nvalue = [40, 4]'),
Text(0.3300970873786408, 0.20833333333333334, 'gini = 0.0\nsamples = 22\nvalue = [22, 0]'),
Text(0.34951456310679613, 0.20833333333333334, 'x[17] <= 42.5\ngini = 0.298\nsamples = 22\nvalue = [18, 4]'),
Text(0.33980582524271846, 0.125, 'x[22] <= 39.0\ngini = 0.463\nsamples = 11\nvalue = [7, 4]'),
Text(0.3300970873786408, 0.041666666666666664, 'gini = 0.444\nsamples = 6\nvalue = [2, 4]'),
Text(0.34951456310679613, 0.041666666666666664, 'gini = 0.0\nsamples = 5\nvalue = [5, 0]'),
Text(0.3592233009708738, 0.125, 'gini = 0.0\nsamples = 11\nvalue = [11, 0]'),
Text(0.34951456310679613, 0.375, 'gini = 0.5\nsamples = 6\nvalue = [3, 3]'),
Text(0.3592233009708738, 0.4583333333333333, 'gini = 0.48\nsamples = 5\nvalue = [2, 3]'),
Text(0.3883495145631068, 0.5416666666666666, 'x[14] <= 41.5\ngini = 0.109\nsamples = 52\nvalue = [49, 3]'),
Text(0.3786407766990291, 0.4583333333333333, 'gini = 0.0\nsamples = 32\nvalue = [32, 0]'),
Text(0.39805825242718446, 0.4583333333333333, 'x[7] <= 76.0\ngini = 0.255\nsamples = 20\nvalue = [17, 3]'),
Text(0.3883495145631068, 0.375, 'x[22] <= 11.0\ngini = 0.444\nsamples = 9\nvalue = [6, 3]'),
Text(0.3786407766990291, 0.2916666666666667, 'gini = 0.0\nsamples = 4\nvalue = [4, 0]'),
Text(0.39805825242718446, 0.2916666666666667, 'gini = 0.48\nsamples = 5\nvalue = [2, 3]'),
Text(0.4077669902912621, 0.375, 'gini = 0.0\nsamples = 11\nvalue = [11,
```

```
0]'),
  Text(0.5224514563106796, 0.7916666666666666, 'x[8] <= 6.5\ngini =
0.277\nsamples = 247\nvalue = [206, 41]'),
  Text(0.4441747572815534, 0.7083333333333334, 'x[8] <= 2.5\ngini =
0.44\nsamples = 52\nvalue = [35, 17]'),
  Text(0.4174757281553398, 0.625, 'x[10] <= 4.5\ngini = 0.198\nsamples =
18\nvalue = [16, 2]'),
  Text(0.4077669902912621, 0.5416666666666666, 'gini = 0.0\nsamples =
13\nvalue = [13, 0]'),
  Text(0.42718446601941745, 0.5416666666666666, 'gini = 0.48\nsamples =
5\nvalue = [3, 2]'),
  Text(0.470873786407767, 0.625, 'x[9] <= 6.5\ngini = 0.493\nsamples =
34\nvalue = [19, 15]'),
  Text(0.44660194174757284, 0.5416666666666666, 'x[16] <= 4.5\ngini =
0.337\nsamples = 14\nvalue = [11, 3]'),
  Text(0.4368932038834951, 0.4583333333333333, 'gini = 0.5\nsamples = 6\nvalue
= [3, 3]'),
  Text(0.4563106796116505, 0.4583333333333333, 'gini = 0.0\nsamples = 8\nvalue
= [8, 0]'),
  Text(0.49514563106796117, 0.5416666666666666, 'x[10] <= 3.5\ngini =
0.48\nsamples = 20\nvalue = [8, 12]'),
  Text(0.47572815533980584, 0.4583333333333333, 'x[14] <= 81.0\ngini =
0.298\nsamples = 11\nvalue = [2, 9]'),
  Text(0.46601941747572817, 0.375, 'gini = 0.0\nsamples = 6\nvalue = [0, 6]'),
  Text(0.4854368932038835, 0.375, 'gini = 0.48\nsamples = 5\nvalue = [2, 3]'),
  Text(0.5145631067961165, 0.4583333333333333, 'x[15] <= 0.5\ngini =
0.444\nsamples = 9\nvalue = [6, 3]'),
  Text(0.5048543689320388, 0.375, 'gini = 0.375\nsamples = 4\nvalue = [1,
3]'),
  Text(0.5242718446601942, 0.375, 'gini = 0.0\nsamples = 5\nvalue = [5, 0]'),
  Text(0.6007281553398058, 0.7083333333333334, 'x[7] <= 26.5\ngini =
0.216\nsamples = 195\nvalue = [171, 24]'),
  Text(0.5533980582524272, 0.625, 'x[14] <= 89.5\ngini = 0.375\nsamples =
32\nvalue = [24, 8]'),
  Text(0.5436893203883495, 0.5416666666666666, 'x[4] <= 1.5\ngini =
0.252\nsamples = 27\nvalue = [23, 4]'),
  Text(0.5339805825242718, 0.4583333333333333, 'gini = 0.0\nsamples =
14\nvalue = [14, 0]'),
  Text(0.5533980582524272, 0.4583333333333333, 'x[14] <= 71.5\ngini =
0.426\nsamples = 13\nvalue = [9, 4]'),
  Text(0.5436893203883495, 0.375, 'gini = 0.375\nsamples = 4\nvalue = [1,
3]'),
  Text(0.5631067961165048, 0.375, 'x[12] <= 3.5\ngini = 0.198\nsamples =
9\nvalue = [8, 1]'),
  Text(0.5533980582524272, 0.2916666666666667, 'gini = 0.0\nsamples = 5\nvalue
= [5, 0]'),
  Text(0.5728155339805825, 0.2916666666666667, 'gini = 0.375\nsamples =
4\nvalue = [3, 1]'),
  Text(0.5631067961165048, 0.5416666666666666, 'gini = 0.32\nsamples =
5\nvalue = [1, 4]'),
  Text(0.6480582524271845, 0.625, 'x[5] <= 3.5\ngini = 0.177\nsamples =
163\nvalue = [147, 16]'),
  Text(0.6019417475728155, 0.5416666666666666, 'x[21] <= 1.5\ngini =
0.095\nsamples = 100\nvalue = [95, 5]'),
```

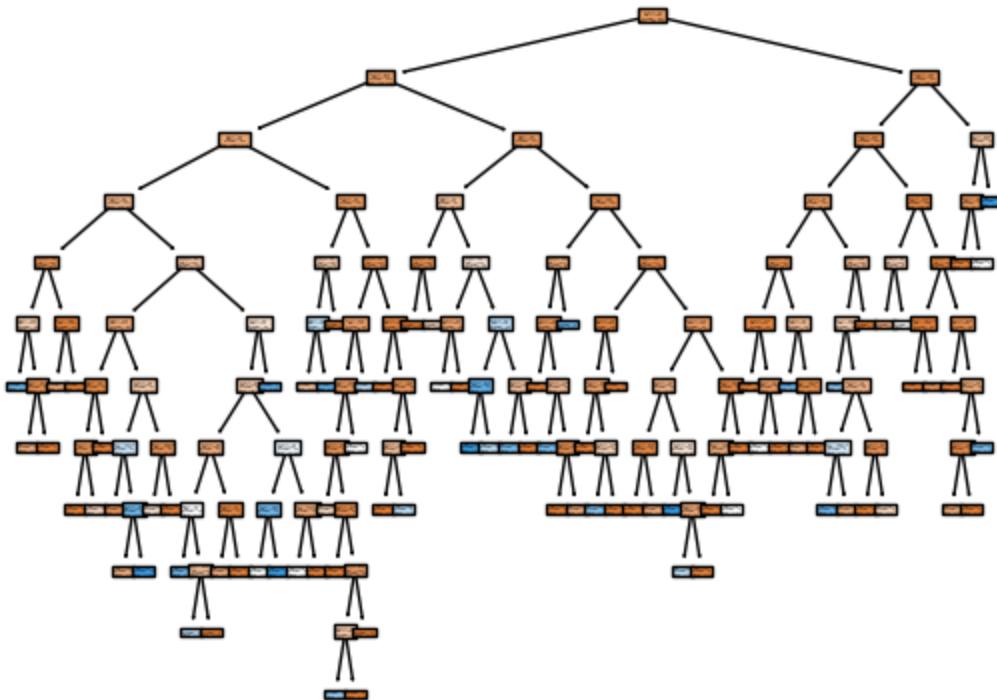
```
Text(0.5922330097087378, 0.4583333333333333, 'x[15] <= 0.5\ngini =
0.32\nsamples = 25\nvalue = [20, 5]'),
Text(0.5825242718446602, 0.375, 'gini = 0.0\nsamples = 12\nvalue = [12,
0]'),
Text(0.6019417475728155, 0.375, 'x[18] <= 12.5\ngini = 0.473\nsamples =
13\nvalue = [8, 5]'),
Text(0.5922330097087378, 0.2916666666666667, 'gini = 0.408\nsamples =
7\nvalue = [2, 5]'),
Text(0.6116504854368932, 0.2916666666666667, 'gini = 0.0\nsamples = 6\nvalue
= [6, 0]'),
Text(0.6116504854368932, 0.4583333333333333, 'gini = 0.0\nsamples =
75\nvalue = [75, 0]'),
Text(0.6941747572815534, 0.5416666666666666, 'x[14] <= 76.5\ngini =
0.288\nsamples = 63\nvalue = [52, 11]'),
Text(0.6601941747572816, 0.4583333333333333, 'x[16] <= 4.5\ngini =
0.404\nsamples = 32\nvalue = [23, 9]'),
Text(0.6407766990291263, 0.375, 'x[22] <= 34.0\ngini = 0.133\nsamples =
14\nvalue = [13, 1]'),
Text(0.6310679611650486, 0.2916666666666667, 'gini = 0.0\nsamples =
10\nvalue = [10, 0]'),
Text(0.6504854368932039, 0.2916666666666667, 'gini = 0.375\nsamples =
4\nvalue = [3, 1]'),
Text(0.6796116504854369, 0.375, 'x[16] <= 5.5\ngini = 0.494\nsamples =
18\nvalue = [10, 8]'),
Text(0.6699029126213593, 0.2916666666666667, 'gini = 0.0\nsamples = 5\nvalue
= [0, 5]'),
Text(0.6893203883495146, 0.2916666666666667, 'x[22] <= 18.0\ngini =
0.355\nsamples = 13\nvalue = [10, 3]'),
Text(0.6796116504854369, 0.2083333333333333, 'gini = 0.48\nsamples =
5\nvalue = [2, 3]'),
Text(0.6990291262135923, 0.2083333333333333, 'gini = 0.0\nsamples =
8\nvalue = [8, 0]'),
Text(0.7281553398058253, 0.4583333333333333, 'x[6] <= 6698.5\ngini =
0.121\nsamples = 31\nvalue = [29, 2]'),
Text(0.7184466019417476, 0.375, 'x[6] <= 4182.5\ngini = 0.375\nsamples =
8\nvalue = [6, 2]'),
Text(0.7087378640776699, 0.2916666666666667, 'gini = 0.0\nsamples = 4\nvalue
= [4, 0]'),
Text(0.7281553398058253, 0.2916666666666667, 'gini = 0.5\nsamples = 4\nvalue
= [2, 2]'),
Text(0.7378640776699029, 0.375, 'gini = 0.0\nsamples = 23\nvalue = [23,
0]'),
Text(0.9235436893203883, 0.875, 'x[18] <= 18.5\ngini = 0.232\nsamples =
247\nvalue = [214, 33]'),
Text(0.7864077669902912, 0.9166666666666667, ' False'),
Text(0.866504854368932, 0.7916666666666666, 'x[22] <= 21.5\ngini =
0.207\nsamples = 230\nvalue = [203, 27]'),
Text(0.8155339805825242, 0.7083333333333333, 'x[12] <= 3.5\ngini =
0.299\nsamples = 104\nvalue = [85, 19]'),
Text(0.7766990291262136, 0.625, 'x[16] <= 7.5\ngini = 0.192\nsamples =
65\nvalue = [58, 7]'),
Text(0.7572815533980582, 0.5416666666666666, 'x[22] <= 17.5\ngini =
0.078\nsamples = 49\nvalue = [47.0, 2.0]'),
Text(0.7475728155339806, 0.4583333333333333, 'gini = 0.0\nsamples =
```

```
38\nvalue = [38, 0]'),
  Text(0.7669902912621359, 0.4583333333333333, 'x[3] <= 1.5\ngini =
0.298\nsamples = 11\nvalue = [9, 2]'),
  Text(0.7572815533980582, 0.375, 'gini = 0.5\nsamples = 4\nvalue = [2, 2]'),
  Text(0.7766990291262136, 0.375, 'gini = 0.0\nsamples = 7\nvalue = [7, 0]'),
  Text(0.7961165048543689, 0.5416666666666666, 'x[18] <= 5.5\ngini =
0.43\nsamples = 16\nvalue = [11, 5]'),
  Text(0.7864077669902912, 0.4583333333333333, 'gini = 0.32\nsamples =
5\nvalue = [1, 4]'),
  Text(0.8058252427184466, 0.4583333333333333, 'x[6] <= 7828.5\ngini =
0.165\nsamples = 11\nvalue = [10, 1]'),
  Text(0.7961165048543689, 0.375, 'gini = 0.375\nsamples = 4\nvalue = [3,
1]'),
  Text(0.8155339805825242, 0.375, 'gini = 0.0\nsamples = 7\nvalue = [7, 0]'),
  Text(0.8543689320388349, 0.625, 'x[9] <= 11.5\ngini = 0.426\nsamples =
39\nvalue = [27, 12]'),
  Text(0.8446601941747572, 0.5416666666666666, 'x[7] <= 32.0\ngini =
0.469\nsamples = 32\nvalue = [20, 12]'),
  Text(0.8349514563106796, 0.4583333333333333, 'gini = 0.408\nsamples =
7\nvalue = [2, 5]'),
  Text(0.8543689320388349, 0.4583333333333333, 'x[11] <= 1.5\ngini =
0.403\nsamples = 25\nvalue = [18.0, 7.0]'),
  Text(0.8349514563106796, 0.375, 'x[6] <= 13835.0\ngini = 0.494\nsamples =
9\nvalue = [4, 5]'),
  Text(0.8252427184466019, 0.2916666666666667, 'gini = 0.32\nsamples =
5\nvalue = [1, 4]'),
  Text(0.8446601941747572, 0.2916666666666667, 'gini = 0.375\nsamples =
4\nvalue = [3, 1]'),
  Text(0.8737864077669902, 0.375, 'x[19] <= 3.5\ngini = 0.219\nsamples =
16\nvalue = [14, 2]'),
  Text(0.8640776699029126, 0.2916666666666667, 'gini = 0.0\nsamples =
11\nvalue = [11, 0]'),
  Text(0.883495145631068, 0.2916666666666667, 'gini = 0.48\nsamples = 5\nvalue
= [3, 2]'),
  Text(0.8640776699029126, 0.5416666666666666, 'gini = 0.0\nsamples = 7\nvalue
= [7, 0]'),
  Text(0.9174757281553398, 0.7083333333333334, 'x[17] <= 31.5\ngini =
0.119\nsamples = 126\nvalue = [118, 8]'),
  Text(0.8932038834951457, 0.625, 'x[18] <= 8.5\ngini = 0.469\nsamples =
8\nvalue = [5, 3]'),
  Text(0.883495145631068, 0.5416666666666666, 'gini = 0.375\nsamples =
4\nvalue = [3, 1]'),
  Text(0.9029126213592233, 0.5416666666666666, 'gini = 0.5\nsamples = 4\nvalue
= [2, 2]'),
  Text(0.941747572815534, 0.625, 'x[10] <= 8.5\ngini = 0.081\nsamples =
118\nvalue = [113, 5]'),
  Text(0.9223300970873787, 0.5416666666666666, 'x[6] <= 18745.5\ngini =
0.025\nsamples = 80\nvalue = [79, 1]'),
  Text(0.912621359223301, 0.4583333333333333, 'gini = 0.0\nsamples = 73\nvalue
= [73, 0]'),
  Text(0.9320388349514563, 0.4583333333333333, 'gini = 0.245\nsamples =
7\nvalue = [6, 1]'),
  Text(0.9611650485436893, 0.5416666666666666, 'x[17] <= 49.0\ngini =
0.188\nsamples = 38\nvalue = [34, 4]'),
```

```

Text(0.9514563106796117, 0.4583333333333333, 'gini = 0.0\nsamples =
22\nvalue = [22, 0]'),
Text(0.970873786407767, 0.4583333333333333, 'x[12] <= 3.5\ngini =
0.375\nsamples = 16\nvalue = [12, 4]'),
Text(0.9611650485436893, 0.375, 'x[4] <= 1.5\ngini = 0.153\nsamples =
12\nvalue = [11, 1]'),
Text(0.9514563106796117, 0.2916666666666667, 'gini = 0.375\nsamples =
4\nvalue = [3, 1]'),
Text(0.970873786407767, 0.2916666666666667, 'gini = 0.0\nsamples = 8\nvalue
= [8, 0]'),
Text(0.9805825242718447, 0.375, 'gini = 0.375\nsamples = 4\nvalue = [1,
3]'),
Text(0.9805825242718447, 0.7916666666666666, 'x[7] <= 80.5\ngini =
0.457\nsamples = 17\nvalue = [11, 6]'),
Text(0.970873786407767, 0.7083333333333334, 'x[13] <= 3.5\ngini =
0.26\nsamples = 13\nvalue = [11, 2]'),
Text(0.9611650485436893, 0.625, 'gini = 0.0\nsamples = 9\nvalue = [9, 0]'),
Text(0.9805825242718447, 0.625, 'gini = 0.5\nsamples = 4\nvalue = [2, 2]'),
Text(0.9902912621359223, 0.7083333333333334, 'gini = 0.0\nsamples = 4\nvalue
= [0, 4]')

```



```

# Obtener el accuracy_score
y_pred_c = tree_clf_c.predict(X_test)
accuracy_c = accuracy_score(y_test, y_pred_c)
print("El accuracy score que se tiene es %s" % accuracy_c)

```

El accuracy score que se tiene es 0.71

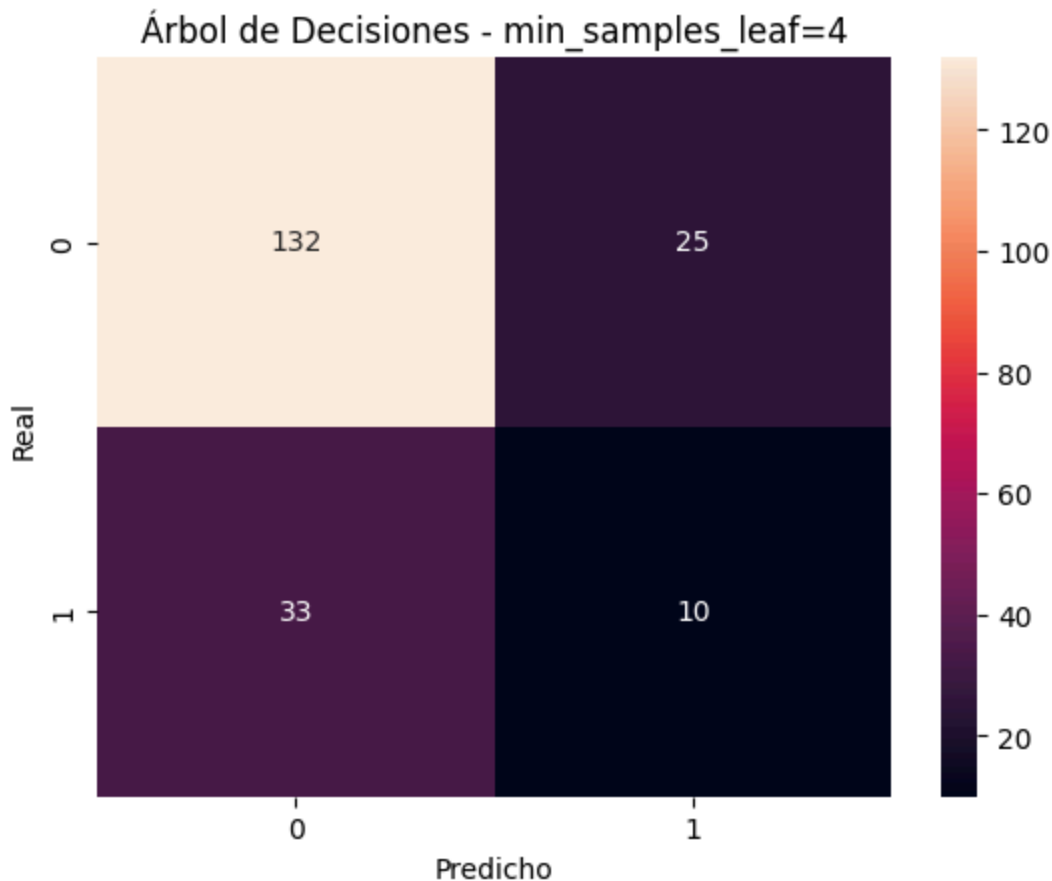
```

# Obtener la matriz de confusión
cm = confusion_matrix(y_test, y_pred_c)
sns.heatmap(cm, annot=True, fmt="d")

```

```
plt.title("Árbol de Decisiones - min_samples_leaf=4")
plt.xlabel("Predicho")
plt.ylabel("Real")
```

```
Text(50.72222222222214, 0.5, 'Real')
```



```
# Obtener la precisión, recall y f1
```

```
precision_c = precision_score(y_test, y_pred_c, average="macro")
```

```
recall_c = recall_score(y_test, y_pred_c, average="macro")
```

```
f1_c = f1_score(y_test, y_pred_c, average="macro")
```

```
metricas = pd.DataFrame({"Métricas": ["Precisión", "Recall", "F1"],
                          "Valores": [precision_c, recall_c, f1_c]})
```

```
metricas
```

	Métricas	Valores
0	Precisión	0.542857
1	Recall	0.536661
2	F1	0.538143

Comparación de los modelos de Árbol de Decisiones

```
comparacion_tree = pd.DataFrame({
    'Modelo': ['Árbol - Defecto', 'Árbol - max_depth=5', 'Árbol - min_samples_leaf=4'],
```

```

'Accuracy': [accuracy_a, accuracy_b, accuracy_c],
'Precisión': [precision_a, precision_b, precision_c],
'Recall': [recall_a, recall_b, recall_c],
'F1': [f1_a, f1_b, f1_c]
})
comparacion_tree

```

	Modelo	Accuracy	Precisión	Recall	F1
0	Árbol - Defecto	0.680	0.564890	0.576655	0.567509
1	Árbol - max_depth=5	0.765	0.475086	0.495704	0.453457
2	Árbol - min_samples_leaf=4	0.710	0.542857	0.536661	0.538143

7. Comparación Global de todos los Algoritmos

A continuación se presenta un resumen consolidado de todos los modelos entrenados, permitiendo identificar el mejor algoritmo para la detección temprana de abandono laboral.

```

comparacion_global = pd.DataFrame({
    'Algoritmo': [
        'Regresión Logística (defecto)',
        'Regresión Logística (L1)',
        'Regresión Logística (C=0.5)',
        'KNN (k=5)',
        'KNN (k=3)',
        'KNN (k=11)',
        'Árbol de Decisión (defecto)',
        'Árbol de Decisión (max_depth=5)',
        'Árbol de Decisión (min_samples_leaf=4)'
    ],
    'Accuracy': [
        accuracy_lr_a, accuracy_lr_b, accuracy_lr_c,
        accuracy_knn_a, accuracy_knn_b, accuracy_knn_c,
        accuracy_a, accuracy_b, accuracy_c
    ],
    'F1 (macro)': [
        f1_lr_a, f1_lr_b, f1_lr_c,
        f1_knn_a, f1_knn_b, f1_knn_c,
        f1_a, f1_b, f1_c
    ]
})

comparacion_global.sort_values('F1 (macro)',
                               ascending=False).reset_index(drop=True)

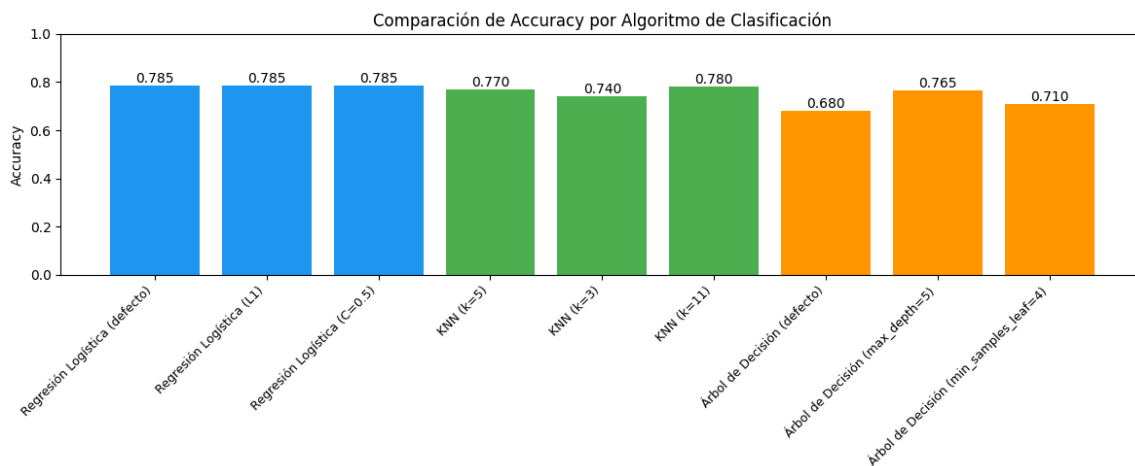
```

	Algoritmo	Accuracy	F1 (macro)
0	Árbol de Decisión (defecto)	0.680	0.567509

	Algoritmo	Accuracy	F1 (macro)
1	Árbol de Decisión (min_samples_leaf=4)	0.710	0.538143
2	KNN (k=3)	0.740	0.475701
3	KNN (k=5)	0.770	0.455492
4	Árbol de Decisión (max_depth=5)	0.765	0.453457
5	Regresión Logística (defecto)	0.785	0.439776
6	Regresión Logística (C=0.5)	0.785	0.439776
7	Regresión Logística (L1)	0.785	0.439776
8	KNN (k=11)	0.780	0.438202

Gráfico comparativo de Accuracy por algoritmo

```
plt.figure(figsize=(12, 5))
colores = ['#2196F3'] * 3 + ['#4CAF50'] * 3 + ['#FF9800'] * 3
barras = plt.bar(range(9), comparacion_global['Accuracy'], color=colores)
for barra in barras:
    altura = barra.get_height()
    plt.text(barra.get_x() + barra.get_width() / 2, altura, f'{altura:.3f}',
             ha='center', va='bottom')
plt.xticks(range(9), comparacion_global['Algoritmo'], rotation=45,
           ha='right', fontsize=9)
plt.ylabel('Accuracy')
plt.title('Comparación de Accuracy por Algoritmo de Clasificación')
plt.ylim(0, 1)
plt.tight_layout()
plt.show()
```



#Comparación global de todas las métricas

```
comparacion_metricas_global = pd.DataFrame({
    'Modelo': [
        'RL - Defecto',
        'RL - L1 (Lasso)',
        'RL - C=0.5, iter=500',
        'KNN - k=5',
        'KNN - k=3',
```

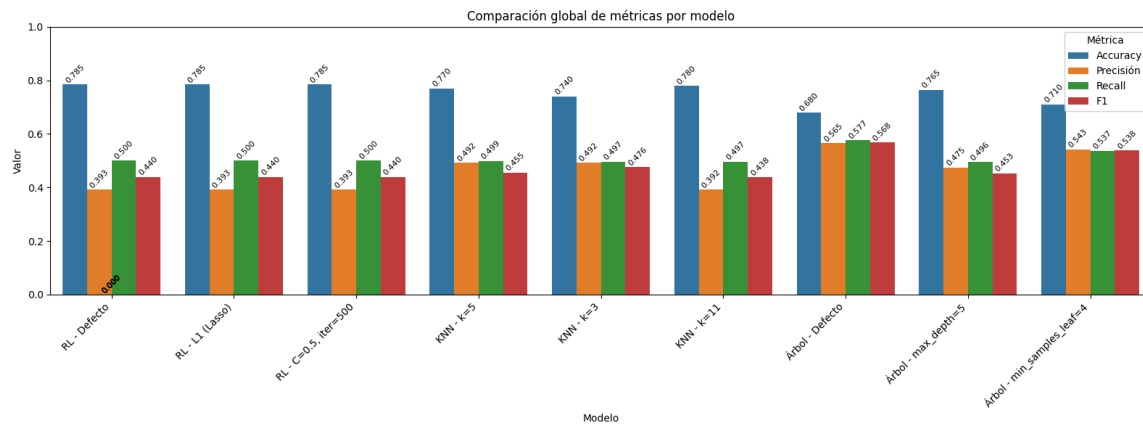
```

        'KNN - k=11',
        'Árbol - Defecto',
        'Árbol - max_depth=5',
        'Árbol - min_samples_leaf=4'
    ],
    'Accuracy': [
        accuracy_lr_a, accuracy_lr_b, accuracy_lr_c,
        accuracy_knn_a, accuracy_knn_b, accuracy_knn_c,
        accuracy_a, accuracy_b, accuracy_c
    ],
    'Precisión': [
        precision_lr_a, precision_lr_b, precision_lr_c,
        precision_knn_a, precision_knn_b, precision_knn_c,
        precision_a, precision_b, precision_c
    ],
    'Recall': [
        recall_lr_a, recall_lr_b, recall_lr_c,
        recall_knn_a, recall_knn_b, recall_knn_c,
        recall_a, recall_b, recall_c
    ],
    'F1': [
        f1_lr_a, f1_lr_b, f1_lr_c,
        f1_knn_a, f1_knn_b, f1_knn_c,
        f1_a, f1_b, f1_c
    ]
})

comparacion_metricas_larga = comparacion_metricas_global.melt(
    id_vars='Modelo',
    var_name='Métrica',
    value_name='Valor'
)

plt.figure(figsize=(16, 6))
ax = sns.barplot(x='Modelo', y='Valor', hue='Métrica',
    data=comparacion_metricas_larga)
for p in ax.patches:
    valor = p.get_height()
    ax.annotate(f'{valor:.3f}',
        (p.get_x() + p.get_width() / 2, valor),
        ha='center', va='bottom', fontsize=8, rotation=45)
plt.title('Comparación global de métricas por modelo')
plt.xlabel('Modelo')
plt.ylabel('Valor')
plt.ylim(0, 1)
plt.xticks(rotation=45, ha='right')
plt.legend(title='Métrica')
plt.tight_layout()
plt.show()

```



8. Conclusiones

Con base en los resultados obtenidos a partir de la aplicación de los tres algoritmos de clasificación (Regresión Logística, KNN y Árbol de Decisiones) sobre el conjunto de datos de abandono laboral, se pueden extraer las siguientes conclusiones:

1. **Regresión Logística:** Ofrece un rendimiento sólido y estable entre sus diferentes configuraciones. La regularización permite controlar el sobreajuste y mejorar la generalización del modelo.
2. **KNN:** Es sensible al valor de k. Valores muy pequeños de k pueden provocar sobreajuste, mientras que valores muy grandes pueden suavizar en exceso las fronteras de decisión. Es importante normalizar los datos antes de aplicar este algoritmo.
3. **Árbol de Decisiones:** Sin restricciones puede sobreajustarse completamente a los datos de entrenamiento. La limitación de la profundidad máxima (max_depth) o del número mínimo de muestras por hoja (min_samples_leaf) ayuda a construir modelos más generalizables.

La selección del mejor modelo debe considerar no solo el accuracy, sino también métricas como F1, precision y recall, especialmente cuando existe desbalance de clases en la variable objetivo Attrition.